

A Neural Probabilistic Language Model

Yoshua Bengio
Réjean Ducharme
Pascal Vincent
Christian Jauvin

BENGIOY@IRO.UMONTREAL.CA
DUCHARME@IRO.UMONTREAL.CA
VINCENTP@IRO.UMONTREAL.CA
JAUVINC@IRO.UMONTREAL.CA

Département d’Informatique et Recherche Opérationnelle
Centre de Recherche Mathématiques
Université de Montréal, Montréal, Québec, Canada

Editors: Jaz Kandola, Thomas Hofmann, Tomaso Poggio and John Shawe-Taylor

Abstract

A goal of statistical language modeling is to learn the joint probability function of sequences of words in a language. This is intrinsically difficult because of the **curse of dimensionality**: a word sequence on which the model will be tested is likely to be different from all the word sequences seen during training. Traditional but very successful approaches based on n-grams obtain generalization by concatenating very short overlapping sequences seen in the training set. We propose to fight the curse of dimensionality by **learning a distributed representation for words** which allows each training sentence to inform the model about an exponential number of semantically neighboring sentences. The model learns simultaneously (1) a distributed representation for each word along with (2) the probability function for word sequences, expressed in terms of these representations. Generalization is obtained because a sequence of words that has never been seen before gets high probability if it is made of words that are similar (in the sense of having a nearby representation) to words forming an already seen sentence. Training such large models (with millions of parameters) within a reasonable time is itself a significant challenge. We report on experiments using neural networks for the probability function, showing on two text corpora that the proposed approach significantly improves on state-of-the-art n-gram models, and that the proposed approach allows to take advantage of longer contexts.

Keywords: Statistical language modeling, artificial neural networks, distributed representation, curse of dimensionality

1. Introduction

A fundamental problem that makes language modeling and other learning problems difficult is the *curse of dimensionality*. It is particularly obvious in the case when one wants to model the joint distribution between many discrete random variables (such as words in a sentence, or discrete attributes in a data-mining task). For example, if one wants to model the joint distribution of 10 consecutive words in a natural language with a vocabulary V of size 100,000, there are potentially $100\,000^{10} - 1 = 10^{50} - 1$ free parameters. When modeling continuous variables, we obtain generalization more easily (e.g. with smooth classes of functions like multi-layer neural networks or Gaussian mixture models) because the function to be learned can be expected to have some local smoothness properties. For discrete spaces, the generalization structure is not as obvious: any change of these discrete variables may have a drastic impact on the value of the function to be esti-

《神经概率语言模型》

约书亚·本吉奥
雷让·杜沙尔姆
帕斯卡尔·文森特
克里斯蒂安·乔万

BENGIOY@蒙特利尔大学IRO实验室邮箱地址
DUCHARME@蒙特利尔大学IRO实验室邮箱地址
VINCENTP@蒙特利尔大学IRO实验室邮箱地址
JAUVINC@IRO.UMONTREAL.CA

加拿大魁北克省蒙特利尔市，蒙特利尔大学数学研究
中心运筹学与计算机科学系

编者：贾兹·坎多拉、托马斯·霍夫曼、托马索·波焦以及约翰·肖威-泰勒

摘要

统计语言建模的一个目标是学习某种语言中单词序列的联合概率函数。由于存在维度灾难，这一任务本质上极具难度——用于测试模型的单词序列很可能与训练过程中出现过的所有单词序列都不同。传统但极为有效的基于n-gram的方法，是通过拼接训练集中出现的极短且重叠的序列来实现泛化的。我们则提出通过为单词学习分布式表示来应对维度灾难，这种表示方式能让每一条训练句子为模型提供大量语义上相近的句子的信息。该模型能够同时学习（1）每个单词的分布式表示，以及（2）用这些表示来表达的单词序列概率函数。这样一来，即便遇到此前从未出现过的单词序列，只要其构成元素在表示层面与已有句子中的单词相似，该序列也能获得较高的概率。而在合理时间内训练出包含数百万个参数的如此大规模模型本身也是一项重大挑战。本文通过实验展示了使用神经网络来构建概率函数的可行性，并在两个文本语料库上的实验结果表明，所提出的方法显著优于现有的最先进n-gram模型，同时还使得模型能够更好地利用更长的上下文信息。

关键词：统计语言建模、人工神经网络、分布式表示、维度灾难

1. 引言

使得语言建模及其他学习问题面临巨大挑战的一个核心问题，便是维度灾难。当人们试图对多个离散随机变量（例如句子中的单词，或是数据挖掘任务中的离散属性）之间的联合分布进行建模时，这一问题尤为突出。举例来说，若要使用规模为100,000的词汇表 V 来建模自然语言中连续10个单词的联合分布，那么潜在的自由参数数量可达 $100\,000^{10} - 1 = 10^{50} - 1$ 个。而在处理连续变量时，我们更容易实现泛化（比如借助多层神经网络或高斯混合模型这类具有平滑特性的函数类），因为这类函数通常具备一定的局部平滑性。但对于离散空间而言，其泛化机制并不那么明显：这些离散变量的任何微小变化都可能对所需估计的函数值产生极其显著的影响。此外，当每个离散变量所能取值的数量很大时，大多数实际观测到的对象在汉明距离上几乎都会处于彼此最远的状态。

mated, and when the number of values that each discrete variable can take is large, most observed objects are almost maximally far from each other in hamming distance.

A useful way to visualize how different learning algorithms generalize, inspired from the view of non-parametric density estimation, is to think of how probability mass that is initially concentrated on the training points (e.g., training sentences) is distributed in a larger volume, usually in some form of neighborhood around the training points. In high dimensions, it is crucial to distribute probability mass where it matters rather than uniformly in all directions around each training point. We will show in this paper that the way in which the approach proposed here generalizes is fundamentally different from the way in which previous state-of-the-art statistical language modeling approaches are generalizing.

A statistical model of language can be represented by the conditional probability of the next word given all the previous ones, since

$$\hat{P}(w_1^T) = \prod_{t=1}^T \hat{P}(w_t | w_1^{t-1}),$$

where w_t is the t -th word, and writing sub-sequence $w_i^j = (w_i, w_{i+1}, \dots, w_{j-1}, w_j)$. Such statistical language models have already been found useful in many technological applications involving natural language, such as speech recognition, language translation, and information retrieval. Improvements in statistical language models could thus have a significant impact on such applications.

When building statistical models of natural language, one considerably reduces the difficulty of this modeling problem by taking advantage of word order, and the fact that temporally closer words in the word sequence are statistically more dependent. Thus, *n*-gram models construct tables of conditional probabilities for the next word, for each one of a large number of *contexts*, i.e. combinations of the last $n - 1$ words:

$$\hat{P}(w_t | w_1^{t-1}) \approx \hat{P}(w_t | w_{t-n+1}^{t-1}).$$

We only consider those combinations of successive words that actually occur in the training corpus, or that occur frequently enough. What happens when a new combination of n words appears that was not seen in the training corpus? We do not want to assign zero probability to such cases, because such new combinations are likely to occur, and they will occur even more frequently for larger context sizes. A simple answer is to look at the probability predicted using a smaller context size, as done in back-off trigram models (Katz, 1987) or in smoothed (or interpolated) trigram models (Jelinek and Mercer, 1980). So, in such models, how is generalization basically obtained from sequences of words seen in the training corpus to new sequences of words? A way to understand how this happens is to think about a generative model corresponding to these interpolated or back-off n -gram models. Essentially, a new sequence of words is generated by “gluing” very short and overlapping pieces of length 1, 2 ... or up to n words that have been seen frequently in the training data. The rules for obtaining the probability of the next piece are implicit in the particulars of the back-off or interpolated n -gram algorithm. Typically researchers have used $n = 3$, i.e. trigrams, and obtained state-of-the-art results, but see Goodman (2001) for how combining many tricks can yield to substantial improvements. Obviously there is much more information in the sequence that immediately precedes the word to predict than just the identity of the previous couple of words. There are at least two characteristics in this approach which beg to be improved upon, and that we

经过配对后，当每个离散变量所能取的值数量较多时，大多数观测对象在汉明距离上的值几乎都处于最大可能值。

一种用于直观展示不同学习算法泛化能力的有效方法源自非参数密度估计的思路，即思考最初集中在训练样本（例如训练句子）上的概率质量是如何在更大的空间中分布的，通常是在这些训练样本周围的某个邻域范围内。在高维空间中，至关重要的是将概率质量分配在真正重要的位置，而非在每个训练样本周围均匀分布。本文将证明，此处提出的方法所展现的泛化方式，与以往最先进的统计语言建模方法的泛化方式有着本质上的不同。

由于语言的统计模型可以通过给定所有先前单词的情况下下一个单词的条件概率来表示，

$$\hat{P}(w_1^T) = \prod_{t=1}^T \hat{P}(w_t | w_1^{t-1}),$$

其中 w_t 表示第 t 个单词，而子序列 $w_{ji} = (w_i, w_{i+1}, \dots, w_{j-1}, w_j)$ 也是如此。这类统计语言模型已在许多涉及自然语言的技术应用中展现出实用性，比如语音识别、语言翻译以及信息检索。因此，统计语言模型的改进有望对这些应用产生重大影响。

在构建自然语言的统计模型时，利用词序这一特性以及序列中时间上更接近的词在统计上具有更强依赖性这一事实，能够大幅降低建模的难度。因此， n 元语法模型会为大量不同上下文——也就是由最后 $n - 1$ 个词构成的各种组合——分别生成下一个词的条件概率表。

$$\hat{P}(w_t | w_1^{t-1}) \approx \hat{P}(w_t | w_{t-n+1}^{t-1}).$$

我们仅考虑那些在训练语料库中确实出现过，或者出现频率足够高的连续词语组合。那么，当出现一种在训练语料库中从未出现过的 n 个词语的新组合时该如何处理呢？我们并不想给这类情况赋予零概率，因为这样的新组合很可能会出现，而且随着上下文长度的增加，其出现频率还会更高。一个简单的解决办法是参考使用更短上下文长度时预测出的概率，正如回退式三元组模型（Katz, 1987）或平滑（或插值）式三元组模型（Jelinek和Mercer, 1980）中所做的那样。那么，在这类模型中，究竟是如何从训练语料库中的词语序列实现向新词语序列的泛化的呢？要理解这一过程，可以思考与这些插值或回退式 n 元语法模型相对应的生成模型。本质上，新的词语序列是通过“拼接”在训练数据中出现频率较高的、长度为1、2……乃至 n 的极短且重叠的词语片段来生成的。而计算下一个片段概率的规则则蕴含在回退式或插值式 n 元语法算法的具体设计之中。通常研究人员会使用 $n = 3$ ，也就是三元组模型，并由此获得最先进的成果，不过关于结合多种技巧如何带来显著改进的内容，可参考Goodman（2001）的研究。显然，用于预测的不仅包括前几个词语的身份，还包括紧接在目标词语之前的整个序列中的大量信息。这种方法至少存在两个亟待改进的缺陷，而这正是我们

will focus on in this paper. First, it is not taking into account contexts farther than 1 or 2 words,¹ second it is not taking into account the “similarity” between words. For example, having seen the sentence “The cat is walking in the bedroom” in the training corpus should help us generalize to make the sentence “A dog was running in a room” almost as likely, simply because “dog” and “cat” (resp. “the” and “a”, “room” and “bedroom”, etc...) have similar semantic and grammatical roles.

There are many approaches that have been proposed to address these two issues, and we will briefly explain in Section 1.2 the relations between the approach proposed here and some of these earlier approaches. We will first discuss what is the basic idea of the proposed approach. A more formal presentation will follow in Section 2, using an implementation of these ideas that relies on shared-parameter multi-layer neural networks. Another contribution of this paper concerns the challenge of training such very large neural networks (with millions of parameters) for very large data sets (with millions or tens of millions of examples). Finally, an important contribution of this paper is to show that training such large-scale model is expensive but feasible, scales to large contexts, and yields good comparative results (Section 4).

Many operations in this paper are in matrix notation, with lower case v denoting a column vector and v' its transpose, A_j the j -th row of a matrix A , and $x.y = x'y$.

1.1 Fighting the Curse of Dimensionality with Distributed Representations

In a nutshell, the idea of the proposed approach can be summarized as follows:

1. associate with each word in the vocabulary a distributed *word feature vector* (a real-valued vector in $\mathbb{R}^m$),
2. express the joint *probability function* of word sequences in terms of the feature vectors of these words in the sequence, and
3. learn simultaneously the *word feature vectors* and the parameters of that *probability function*.

The feature vector represents different aspects of the word: each word is associated with a point in a vector space. The number of features (e.g. $m=30, 60$ or 100 in the experiments) is much smaller than the size of the vocabulary (e.g. $17,000$). The probability function is expressed as a product of conditional probabilities of the next word given the previous ones, (e.g. using a multi-layer neural network to predict the next word given the previous ones, in the experiments). This function has parameters that can be iteratively tuned in order to **maximize the log-likelihood of the training data** or a regularized criterion, e.g. by adding a weight decay penalty.² The feature vectors associated with each word are learned, but they could be initialized using prior knowledge of semantic features.

Why does it work? In the previous example, if we knew that dog and cat played similar roles (semantically and syntactically), and similarly for (the,a), (bedroom,room), (is,was),

1. n-grams with n up to 5 (i.e. 4 words of context) have been reported, though, but due to data scarcity, most predictions are made with a much shorter context.
2. Like in ridge regression, the squared norm of the parameters is penalized.

将在本文中重点探讨的方面。首先，这类模型没有考虑超出1或2个词语之外的上下文；¹其次，它也没有考虑词语之间的“相似性”。例如，如果在训练语料库中见过“猫正在卧室里行走”这句话，就应该有助于我们实现泛化，使得“一只狗正在房间里奔跑”这句话的出现概率也变得相当高，仅仅是因为“狗”和“猫”（以及“the”和“a”、“房间”和“卧室”等）在语义和语法功能上具有相似性。

为了解决这两个问题，已有许多不同的研究方法被提出。我们将在1.2节简要阐述此处提出的方法与早期部分方法之间的关系。首先会介绍该方法核心理念，随后在第2节中，我们将借助基于共享参数多层神经网络的实现方式，对这一理念进行更正式的阐述。本文的另一个贡献在于应对训练规模极大的神经网络（包含数百万个参数）处理海量数据集（包含数百万乃至数千万条样本）所带来的挑战。最后，本文的一个重要成果是证明了尽管训练如此大规模的模型成本高昂，但仍然是可行的，而且这类模型能够处理较长的上下文，并能取得良好的对比效果（详见第4节）。

本文中的许多运算都采用矩阵表示法：小写的 v 表示列向量， v' 则表示其转置； A_j 表示矩阵 A 的第 j 行，而 $x.y = x'y$ 则表示相应的运算结果。

1.1 利用分布式表示对抗维度灾难

简而言之，所提出方法的核心思想可概括为以下几点：

1. 为词汇表中的每个单词关联一个分布式词特征向量（即属于 $\mathbb{R}^m$ 范围内的实数向量）；
2. 以序列中各单词的词特征向量为基础，表达词序列的联合概率函数；
3. 同时学习这些词特征向量以及该概率函数的各类参数。

词特征向量用于表征单词的不同属性：每个单词都会对应向量空间中的一个点，其特征数量（例如实验中的 $m=30$ ，可为60或100）远小于词汇表的规模（例如17,000）。该概率函数表现为给定前序单词条件下下一个单词的条件概率的乘积形式（实验中会采用多层神经网络来预测给定前序单词后的下一个单词）。该函数包含若干参数，可通过迭代优化的方式调整，从而最大化训练数据的对数似然值或满足某种正则化准则，比如通过添加权重衰减惩罚来实现。² 虽然每个单词的词特征向量是经过学习得到的，但也可以借助关于语义特征的先验知识对它们进行初始化。

为何它能有效工作？在之前的示例中，如果我们知道‘dog’和‘cat’在语义和句法层面扮演着相似的角色，而‘(the,a)’、‘(bedroom,room)’、‘(is,was)’以及‘(running,walking)’也是如此，那么我们就能够自然而然地对概率质量进行泛化（即迁移）。

1. 目前已有研究报道过使用长度为5的n元语法模型（即包含4个上下文词的模型），但由于数据量不足，大多数预测实际上是在更短的上下文长度下进行的。2. 与岭回归类似，这类模型也会对参数的平方范数施加惩罚。

(running,walking), we could naturally generalize (i.e. transfer probability mass) from

	The cat is walking in the bedroom
to	A dog was running in a room
and likewise to	The cat is running in a room
	A dog is walking in a bedroom
	The dog was walking in the room
	...

and many other combinations. In the proposed model, it will so generalize because “similar” words are expected to have a similar feature vector, and because the probability function is a *smooth* function of these feature values, a small change in the features will induce a small change in the probability. Therefore, the presence of only one of the above sentences in the training data will increase the probability, not only of that sentence, but also of its combinatorial number of “neighbors” in sentence space (as represented by sequences of feature vectors).

1.2 Relation to Previous Work

The idea of using neural networks to model high-dimensional discrete distributions has already been found useful to learn the joint probability of $Z_1 \cdots Z_n$, a set of random variables where each is possibly of a different nature (Bengio and Bengio, 2000a,b). In that model, the joint probability is decomposed as a product of conditional probabilities

$$\hat{P}(Z_1 = z_1, \cdots, Z_n = z_n) = \prod_i \hat{P}(Z_i = z_i | g_i(Z_{i-1} = z_{i-1}, Z_{i-2} = z_{i-2}, \cdots, Z_1 = z_1)),$$

where $g(.)$ is a function represented by a neural network with a special left-to-right architecture, with the i -th output block $g_i()$ computing parameters for expressing the conditional distribution of Z_i given the value of the previous Z ’s, in some arbitrary order. Experiments on four UCI data sets show this approach to work comparatively very well (Bengio and Bengio, 2000a,b). Here we must deal with data of variable length, like sentences, so the above approach must be adapted. Another important difference is that here, all the Z_i (word at i -th position), refer to the same type of object (a word). The model proposed here therefore introduces a sharing of parameters across time – the same g_i is used across time – that is, and across input words at different positions. It is a successful large-scale application of the same idea, along with the (old) idea of learning a distributed representation for symbolic data, that was advocated in the early days of connectionism (Hinton, 1986, Elman, 1990). More recently, Hinton’s approach was improved and successfully demonstrated on learning several symbolic relations (Paccanaro and Hinton, 2000). The idea of using neural networks for language modeling is not new either (e.g. Miikkulainen and Dyer, 1991). In contrast, here we push this idea to a **large scale**, and concentrate on learning a **statistical model** of the distribution of word sequences, rather than learning the role of words in a sentence. The approach proposed here is also related to previous proposals of character-based text compression using neural networks to predict the probability of the next character (Schmidhuber, 1996). The idea of using a neural network for language modeling has also been independently proposed by Xu and Rudnicky (2000), although experiments are with networks without hidden units and a single input word, which limit the model to essentially capturing unigram and bigram statistics.

The idea of discovering some similarities between words to obtain generalization from training sequences to new sequences is not new. For example, it is exploited in approaches that are based on learning a clustering of the words (Brown et al., 1992, Pereira et al., 1993, Niesler et al., 1998, Baker

也就是说，我们可以将概率质量从一种情况迁移到另一种情况。

	那只猫正在卧室里行走。	m
to	有一只狗在某个房间里奔跑。	
同样地，也适用于...	那只猫在某个房间里奔跑，一只狗在	
...	卧室里行走，那只狗也在同一个房间	
	里行走……	

此外还存在许多其他组合情况。在所提出的模型中，该模型能够实现良好的泛化能力，这是因为人们认为“相似”的词语应当具有相似的特征向量；同时由于概率函数是这些特征值的平滑函数，特征上的微小变化只会引起概率的微小变动。因此，训练数据中仅出现上述句子中的某一句，不仅会提升该句子本身的概率，还会提升其在句子空间中所有“邻近”句子的概率——这些邻近句子可通过特征向量的序列来表示，其数量由组合数决定。

1.2 与先前研究的关系

利用神经网络对高维离散分布进行建模的理念已被证实有助于计算随机变量集合 $Z_1 \cdots Z_n$ 的联合概率，这些随机变量的类型可能各不相同（Bengio和Bengio，2000a,b）。在该模型中，联合概率被分解为多个条件概率的乘积。

$$\hat{P}(Z_1 = z_1, \cdots, Z_n = z_n) = \prod_i \hat{P}(Z_i = z_i | g_i(Z_{i-1} = z_{i-1}, Z_{i-2} = z_{i-2}, \cdots, Z_1 = z_1)),$$

其中 $g(.)$ 是由具有特殊从左到右架构的神经网络所表示的函数，其第 i 个输出模块 $g_i()$ 负责计算参数，用以根据之前各个 Z 的值——按某种任意顺序——来表达 Z_i 的条件分布。在四个UCI数据集上进行的实验表明，这种方法的效果相当不错（Bengio和Bengio，2000a,b）。由于此处需要处理长度可变的文本，如句子，因此必须对上述方法进行调整。另一个重要区别在于，此处的所有 Z_i （即第 i 位上的单词）都属于同一类对象——单词。因此，这里提出的模型引入了参数在时间维度上的共享机制，即同一个 g_i 会在不同时间以及不同位置的输入单词之间被重复使用。这实际上是同一理念在更大规模上的成功应用，同时也延续了连接主义早期就倡导的为符号数据学习分布式表示的思路（Hinton, 1986；Elman, 1990）。更为近期的是，Hinton的方法得到了改进，并已成功用于学习多种符号关系（Paccanaro和Hinton，2000）。利用神经网络进行语言建模的理念也并非新事物（例如Miikkulainen和Dyer，1991）。不同之处在于，此处我们将这一理念推广到了更大规模，重点在于构建词序列分布的统计模型，而非仅研究单词在句子中的功能。此外，这里提出的方法也与以往利用神经网络预测下一个字符概率、从而实现基于字符的文本压缩的方案有关（Schmidhuber, 1996）。Xu和Rudnicky（2000）也独立提出了使用神经网络进行语言建模的想法，不过他们的实验所使用的神经网络没有隐藏单元且仅包含一个输入单词，这限制了该模型只能捕捉单词模型和二元词组模型的统计特征。

从训练序列中发现词语之间的某些相似性，从而实现向新序列的泛化，这一思路并非新鲜事物。例如，在那些基于学习词语聚类的方法中就运用了这一理念（Brownet等人，1992年；Pereira等人，1993年；尼斯勒等人，1998年；Baker）。

and McCallum, 1998): each word is associated deterministically or probabilistically with a discrete class, and words in the same class are similar in some respect. In the model proposed here, instead of characterizing the similarity with a discrete random or deterministic variable (which corresponds to a soft or hard partition of the set of words), we use a continuous real-vector for each word, i.e. a **learned distributed feature vector**, to represent similarity between words. The experimental comparisons in this paper include results obtained with class-based n-grams (Brown et al., 1992, Ney and Kneser, 1993, Niesler et al., 1998).

The idea of using a vector-space representation for words has been well exploited in the area of *information retrieval* (for example see work by Schutze, 1993), where feature vectors for words are learned on the basis of their probability of co-occurring in the same documents (Latent Semantic Indexing, see Deerwester et al., 1990). An important difference is that here we look for a representation for words that is helpful in representing compactly the probability distribution of word sequences from natural language text. Experiments suggest that learning jointly the representation (word features) and the model is very useful. We tried (unsuccessfully) using as fixed word features for each word w the first principal components of the co-occurrence frequencies of w with the words occurring in text around the occurrence of w . This is similar to what has been done with documents for information retrieval with LSI. The idea of using a continuous representation for words has however been exploited successfully by Bellegarda (1997) in the context of an n-gram based statistical language model, using LSI to dynamically identify the topic of discourse.

The idea of a vector-space representation for symbols in the context of neural networks has also previously been framed in terms of a parameter sharing layer, (e.g. Riis and Krogh, 1996) for secondary structure prediction, and for text-to-speech mapping (Jensen and Riis, 2000).

2. A Neural Model

The training set is a sequence $w_1 \cdots w_T$ of words $w_t \in V$, where the vocabulary V is a large but finite set. The objective is to learn a good model $f(w_t, \cdots, w_{t-n+1}) = \hat{P}(w_t | w_1^{t-1})$, in the sense that it gives high out-of-sample likelihood. Below, we report the geometric average of $1/\hat{P}(w_t | w_1^{t-1})$, also known as *perplexity*, which is also the exponential of the average negative log-likelihood. The only constraint on the model is that for any choice of w_1^{t-1} , $\sum_{i=1}^{|V|} f(i, w_{t-1}, \cdots, w_{t-n+1}) = 1$, with $f > 0$. By the product of these conditional probabilities, one obtains a model of the joint probability of sequences of words.

We decompose the function $f(w_t, \cdots, w_{t-n+1}) = \hat{P}(w_t | w_1^{t-1})$ in two parts:

1. A mapping C from any element i of V to a real vector $C(i) \in \mathbb{R}^m$. It represents the *distributed feature vectors* associated with each word in the vocabulary. In practice, C is represented by a $|V| \times m$ matrix of free parameters.
2. The probability function over words, expressed with C : a function g maps an input sequence of feature vectors for words in context, $(C(w_{t-n+1}), \cdots, C(w_{t-1}))$, to a conditional probability distribution over words in V for the next word w_t . The output of g is a vector whose i -th element estimates the probability $\hat{P}(w_t = i | w_1^{t-1})$ as in Figure 1.

$$f(i, w_{t-1}, \cdots, w_{t-n+1}) = g(i, C(w_{t-1}), \cdots, C(w_{t-n+1}))$$

The function f is a composition of these two mappings (C and g), with C being *shared* across all the words in the context. With each of these two parts are associated some parameters. The

在这些基于类别的n元语法模型中，每个单词同样会被确定性地或概率性地关联到某个离散类别，且同一类别内的单词在特定属性上存在相似性。与之不同，本文提出的模型采用连续的实数向量——也就是经过学习得到的分布式特征向量——来表示单词间的相似度，而非依赖离散的随机或确定性变量来实现相似性表征。此外，本文的实验对比还涵盖了使用基于类别的n元语法模型所得出的结果（布朗等人，1992年；内伊与克内瑟，1993年；尼斯勒等人，1998年）。

在信息检索领域，人们早已广泛运用向量空间表示法来处理词汇（例如可参考舒策1993年的研究），这类方法会根据词汇在相同文档中共现的概率来学习对应的特征向量（即潜在语义索引技术，相关研究可见迪尔韦斯特等人1990年的成果）。不过与前述方法的重要区别在于，此处我们寻求的词汇表示方式能够更简洁地呈现自然语言文本中所含词序列的概率分布。实验表明，同时学习词汇的表示形式（即词特征）以及相应的模型具有极大的实用价值。我们曾尝试将每个词汇 w 与其周围文本中出现的其他词汇的共现频率的第一主成分作为固定的词特征，但并未取得理想效果。这一做法其实与信息检索领域中使用LSI技术处理文档时的思路类似。而早在1997年，贝莱加达就在基于n元语法的统计语言模型框架下成功采用了词汇的连续表示方式，并借助LSI技术动态识别文本的主题。

在神经网络的背景下，用于表示符号的向量空间概念此前也曾被用于二级结构预测以及文本转语音映射领域，其实现方式便是通过参数共享层（例如里斯与克罗格在1996年的研究）， $\{v_1\}$ 此外也有相关研究将此类方法应用于文本转语音映射任务（詹森与里斯于2000年发表的研究）。

2. 一个神经网络模型

训练集是由一系列单词序列 $w_1 \cdots w_T$ 构成的，其中每个单词都属于词汇表 V ，而该词汇表 V 虽规模庞大但仍是有限集合。我们的目标是学习一个优秀的模型 $f(w_t, \cdots, w_{t-n+1}) = \hat{P}(w_t | w_1^{t-1})$ ，使得该模型能够获得较高的样本外似然值。下文我们将给出 $1/\hat{P}(w_t | w_1^{t-1})$ 的几何平均值，这一数值也被称为困惑度，实际上它等于平均负对数似然的指数形式。对该模型的唯一约束是：对于任意选定的 w_1^{t-1} ， $\sum_{i=1}^{|V|} f(i, w_{t-1}, \cdots, w_{t-n+1}) = 1$ ，这里的 $f > 0$ 。通过这些条件概率的乘积，就可以构建出描述单词序列联合概率的模型。

我们将函数 $f(w_t, \cdots, w_{t-n+1}) = \hat{P}(w_t | w_1^{t-1})$ 分解为两个部分：

1. 一种映射 C ，它将词汇表 V 中的任意元素 i 映射到一个实数向量 $C(i) \in \mathbb{R}^m$ 。该映射用于表示词汇表中每个单词所对应的分布式特征向量。在实际应用中， C 通常由一个包含 $|V| \times m$ 个自由参数的矩阵来表示。
2. 基于 C 表达的单词概率函数：有一个函数 g ，它能将上下文中各单词的特征向量序列 $(C(w_{t-n+1}), \cdots, C(w_{t-1}))$ 映射为词汇表 V 中下一个单词 w_t 的条件概率分布。 g 的输出是一个向量，其第 i 个元素用于估计如图1所示的概率 $\hat{P}(w_t = i | w_1^{t-1})$ 。

$$f(i, w_{t-1}, \cdots, w_{t-n+1}) = g(i, C(w_{t-1}), \cdots, C(w_{t-n+1}))$$

函数 f 是由这两个映射（ C 和 g ）组合而成的，其中映射 C 会在上下文中的所有词语之间共享。这两个组成部分各自都对应一些参数。

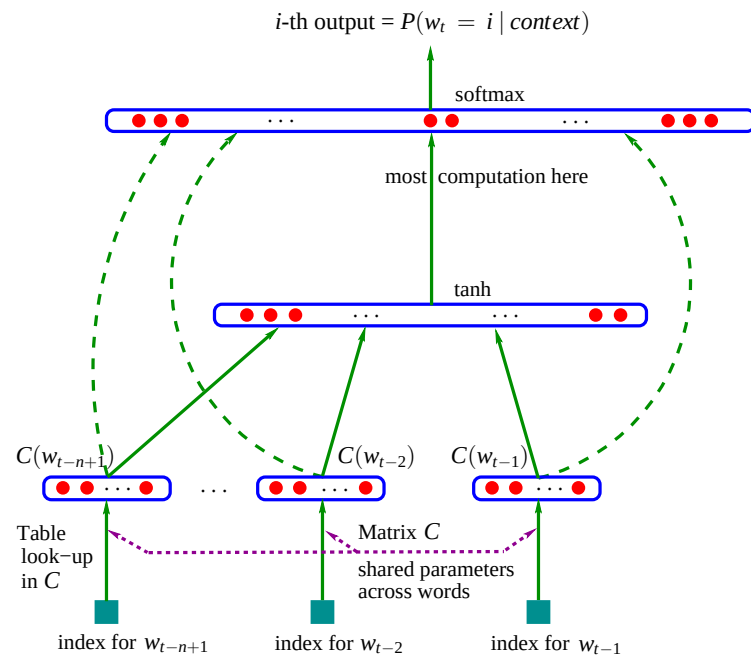


Figure 1: Neural architecture: $f(i, w_{t-1}, \dots, w_{t-n+1}) = g(i, C(w_{t-1}), \dots, C(w_{t-n+1}))$ where g is the neural network and $C(i)$ is the i -th word feature vector.

parameters of the mapping C are simply the feature vectors themselves, represented by a $|V| \times m$ matrix C whose row i is the feature vector $C(i)$ for word i . The function g may be implemented by a feed-forward or recurrent neural network or another parametrized function, with parameters ω . The overall parameter set is $\theta = (C, \omega)$.

Training is achieved by looking for θ that maximizes the training corpus penalized log-likelihood:

$$L = \frac{1}{T} \sum_t \log f(w_t, w_{t-1}, \dots, w_{t-n+1}; \theta) + R(\theta),$$

where $R(\theta)$ is a regularization term. For example, in our experiments, R is a weight decay penalty applied only to the weights of the neural network and to the C matrix, not to the biases.³

In the above model, the number of free parameters **only scales linearly** with V , the number of words in the vocabulary. It also **only scales linearly** with the order n : the scaling factor could be reduced to sub-linear if more sharing structure were introduced, e.g. using a time-delay neural network or a recurrent neural network (or a combination of both).

In most experiments below, the neural network has one hidden layer beyond the word features mapping, and optionally, direct connections from the word features to the output. Therefore there are really two hidden layers: the shared word features layer C , which has no non-linearity (it would not add anything useful), and the ordinary hyperbolic tangent hidden layer. More precisely, the neural network computes the following function, with a *softmax* output layer, which guarantees positive probabilities summing to 1:

$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1}) = \frac{e^{y_{w_t}}}{\sum_i e^{y_i}}.$$

3. The *biases* are the additive parameters of the neural network, such as b and d in equation 1 below.

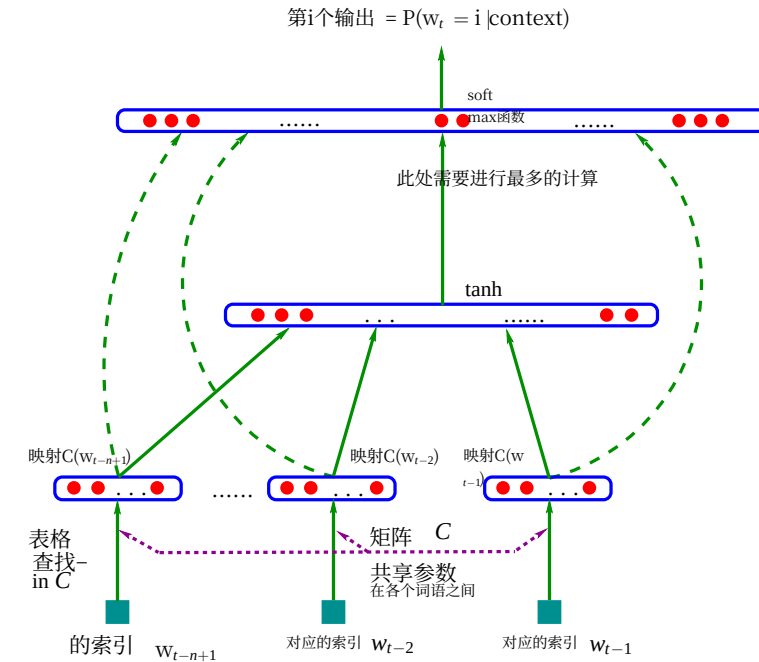


图1: 神经网络结构: $f(i, w_{t-1}, \dots, w_{t-n+1}) = g(i, C(w_{t-1}), \dots, C(w_{t-n+1}))$, 其中 g 代表神经网络, $C(i)$ 为第 i 个词的特征向量。

映射 C 的参数其实就是那些特征向量本身, 它们由一个 $|V| \times m$ 矩阵 C 表示, 该矩阵的第 i 行即为第 i 个词语的 $C(i)$ 特征向量。函数 g 则可以通过前馈神经网络、循环神经网络或其他带参数的函数来实现, 这些函数同样拥有各自的参数。整个模型的参数集合为 $\theta = (C, \omega)$ 。

训练 θ 通过寻找能够最大化经过正则化处理的训练语料库对数似然值的解来获得最优结果。

$$L = \frac{1}{T} \sum_t \log f(w_t, w_{t-1}, \dots, w_{t-n+1}; \theta) + R(\theta),$$

此处 $R(\theta)$ 即为正则化项。例如, 在我们的实验中, R 表现为一种权重衰减惩罚, 仅作用于神经网络的权重以及 C 矩阵的元素, 而不涉及偏置参数。³

在上述模型中, 自由参数的数量仅与词汇表中的词数 V 呈线性增长关系, 同时也仅与阶数 n 呈线性增长关系; 如果引入更多的共享结构, 比如采用时延神经网络或循环神经网络 (或是两者的组合), 这一增长系数甚至可以降为亚线性水平。

在以下大多数实验中, 该神经网络在词特征映射之后还包含一个隐藏层, 此外可选地还会存在从词特征直接连接到输出层的结构。因此实际上存在两个隐藏层: 一个是没有非线性处理的共享词特征层 C (因为非线性处理并不会带来任何实用价值), 另一个则是常规的双曲正切隐藏层。更准确地说, 该神经网络会通过带有softmax输出层的结构来计算相应的函数, 这样的设计能够确保所有概率值之和为1。

$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1}) = \frac{e^{y_{w_t}}}{\sum_i e^{y_i}}.$$

3. 所谓的偏置参数, 指的是神经网络中的加性参数, 例如下式1中所展示的 b 和 d 。

The y_i are the unnormalized log-probabilities for each output word i , computed as follows, with parameters b, W, U, d and H :

$$y = b + Wx + U \tanh(d + Hx) \quad (1)$$

where the hyperbolic tangent $\tanh$ is applied element by element, W is optionally zero (no direct connections), and x is the word features layer activation vector, which is the concatenation of the input word features from the matrix C :

$$x = (C(w_{t-1}), C(w_{t-2}), \dots, C(w_{t-n+1})).$$

Let h be the number of hidden units, and m the number of features associated with each word. When no direct connections from word features to outputs are desired, the matrix W is set to 0. The free parameters of the model are the output biases b (with $|V|$ elements), the hidden layer biases d (with h elements), the hidden-to-output weights U (a $|V| \times h$ matrix), the word features to output weights W (a $|V| \times (n-1)m$ matrix), the hidden layer weights H (a $h \times (n-1)m$ matrix), and the word features C (a $|V| \times m$ matrix):

$$\theta = (b, d, W, U, H, C).$$

The number of free parameters is $|V|(1 + nm + h) + h(1 + (n-1)m)$. The dominating factor is $|V|(nm + h)$. Note that in theory, if there is a weight decay on the weights W and H but not on C , then W and H could converge towards zero while C would blow up. In practice we did not observe such behavior when training with stochastic gradient ascent.

Stochastic gradient ascent on the neural network consists in performing the following iterative update after presenting the t -th word of the training corpus:

$$\theta \leftarrow \theta + \varepsilon \frac{\partial \log \hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})}{\partial \theta}$$

where ε is the “learning rate”. Note that a large fraction of the parameters needs not be updated or visited after each example: the word features $C(j)$ of all words j that do not occur in the input window.

Mixture of models. In our experiments (see Section 4) we have found improved performance by combining the probability predictions of the neural network with those of an interpolated trigram model, either with a simple fixed weight of 0.5, a learned weight (maximum likelihood on the validation set) or a set of weights that are conditional on the frequency of the context (using the same procedure that combines trigram, bigram, and unigram in the interpolated trigram, which is a mixture).

3. Parallel Implementation

Although the number of parameters scales nicely, i.e. linearly with the size of the input window and linearly with the size of the vocabulary, the amount of computation required for obtaining the output probabilities is much greater than that required from n -gram models. The main reason is that with n -gram models, obtaining a particular $P(w_t | w_{t-1}, \dots, w_{t-n+1})$ does not require the computation of the probabilities for all the words in the vocabulary, because of the easy normalization (performed when training the model) enjoyed by the linear combinations of relative frequencies. The main computational bottleneck with the neural implementation is the computation of the activations of the output layer.

y_i 表示每个输出词*i*对应的未归一化的对数概率，其计算方式如下，其中涉及的参数包括b、W、U、d以及H。

$$y = b + Wx + U \tanh(d + Hx) \quad (1)$$

在该模型中，双曲正切函数tanh会逐元素被应用，权重矩阵W可选择设为零（即不存在直接连接），而x则是词特征层激活向量，它由矩阵C中的输入词特征拼接而成。

$$x = (C(w_{t-1}), C(w_{t-2}), \dots, C(w_{t-n+1})).$$

设h为隐藏单元数量，m为每个单词对应的特征数量。当不需要在词特征与输出之间建立直接连接时，权重矩阵W会被设置为0。该模型的自由参数包括：输出偏置参数b（共有 $|V|$ 个元素）、隐藏层偏置参数d（共h个元素）、隐藏层到输出层的权重矩阵U（为一个 $|V| \times h$ 大小的矩阵）、词特征到输出层的权重矩阵W（为一个 $|V| \times (n-1)m$ 大小的矩阵）、隐藏层权重矩阵H（为一个 $h \times (n-1)m$ 大小的矩阵），以及词特征矩阵C（为一个 $|V| \times m$ 大小的矩阵）。

$$\theta = (b, d, W, U, H, C).$$

自由参数的总数为 $|V|(1 + nm + h) + h(1 + (n-1)m)$ 。其中起主导作用的因素是 $|V|(nm + h)$ 。需要注意的是，从理论上讲，如果对权重W和H应用权重衰减而不对映射C应用该策略，那么W和H可能会收敛到零值，而C的值则会变得无限大。但在实际使用随机梯度上升法进行训练时，我们并未观察到此类现象。

在神经网络中，随机梯度上升法指的是在呈现训练语料库中的第t个单词之后，执行以下迭代更新步骤。

$$\theta \leftarrow \theta + \varepsilon \frac{\partial \log \hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})}{\partial \theta}$$

这里的 ε “学习率”指的就是用于控制更新幅度的参数。需要指出的是，并非所有参数都需要在处理每个样本后都进行更新或访问——那些未出现在输入窗口中的所有单词j对应的词特征C(j)就无需处理。

模型混合方法。在我们的实验中（详见第4节），我们发现将神经网络的概率预测结果与经过插值的三元组模型的预测结果相结合，能够提升性能。这种组合方式可以采用固定的0.5权重、基于验证集最大似然算法得到的权重，或是根据上下文出现频率动态调整的权重集合；后者实际上就是一种结合了三元组模型、二元词组模型以及单词模型的模型混合方法。

3. 并行实现方式

尽管该模型的参数数量能够保持良好的增长趋势，即与输入窗口的大小以及词汇表的大小呈线性关系，但为计算输出概率所需的运算量却远高于n元语法模型。其主要原因在于，对于n元语法模型而言，由于相对频率的线性组合在模型训练阶段即可轻松完成归一化处理，因此计算特定概率 $P(w_t | w_{t-1}, \dots, w_{t-n+1})$ 时无需对词汇表中的所有单词逐一计算概率。而在基于神经网络的实现方式中，主要的计算瓶颈则在于输出层激活值的计算。

Running the model (both training and testing) on a parallel computer is a way to reduce computation time. We have explored parallelization on two types of platforms: shared-memory processor machines and Linux clusters with a fast network.

3.1 Data-Parallel Processing

In the case of a shared-memory processor, parallelization is easily achieved, thanks to the very low communication overhead between processors, through the shared memory. In that case we have chosen a data-parallel implementation in which **each processor works on a different subset of the data**. Each processor computes the gradient for its examples, and performs stochastic gradient updates on the parameters of the model, which are simply stored in a shared-memory area. Our first implementation was extremely slow and relied on synchronization commands to make sure that each processor would not write at the same time as another one in one of the above parameter subsets. Most of the cycles of each processor were spent waiting for another processor to release a lock on the write access to the parameters.

Instead we have chosen an **asynchronous implementation** where each processor can write at any time in the shared-memory area. Sometimes, part of an update on the parameter vector by one of the processors is lost, being overwritten by the update of another processor, and this introduces a bit of noise in the parameter updates. However, this noise seems to be very small and did not apparently slow down training.

Unfortunately, large shared-memory parallel computers are very expensive and their processor speed tends to lag behind mainstream CPUs that can be connected in clusters. We have thus been able to obtain much faster training on fast network clusters.

3.2 Parameter-Parallel Processing

If the parallel computer is a network of CPUs, we generally can't afford to frequently exchange all the parameters among the processors, because that represents tens of megabytes (almost 100 megabytes in the case of our largest network), which would take too much time through a local network. Instead we have chosen to **parallelize across the parameters**, in particular the parameters of the output units, because that is where the vast majority of the computation is taking place, in our architecture. Each CPU is responsible for the computation of the unnormalized probability for a subset of the outputs, and performs the updates for the corresponding output unit parameters (weights going into that unit). This strategy allowed us to perform a **parallelized stochastic gradient ascent** with a negligible communication overhead. The CPUs essentially need to communicate two informations: (1) the normalization factor of the output softmax, and (2) the gradients on the hidden layer (denoted a below) and word feature layer (denoted x). All the CPUs duplicate the computations that precede the computation of the output units activations, i.e., the selection of word features and the computation of the hidden layer activation a , as well as the corresponding back-propagation and update steps. However, these computations are a negligible part of the total computation for our networks.

For example, consider the following architecture used in the experiments on the AP (Associated Press) news data: the vocabulary size is $|V| = 17,964$, the number of hidden units is $h = 60$, the order of the model is $n = 6$, the number of word features is $m = 100$. The total number of numerical operations to process a single training example is approximately $|V|(1 + nm + h) + h(1 + nm) + nm$ (where the terms correspond respectively to the computations of the output units, hidden units, and word

在并行计算机上运行该模型（包括训练和测试阶段），是缩短计算时间的有效途径。我们已在两种类型的平台上进行了并行化尝试：具备共享内存处理器的机器，以及配备了高速网络的Linux集群。

3.1 数据并行处理

在采用共享内存处理器的架构中，由于处理器之间通过共享内存进行数据传输的通信开销极低，因此很容易实现并行化。在这种架构下，我们选择了数据并行实现方式，即让每个处理器分别处理数据中的不同子集。每个处理器都会为其处理的样本计算梯度，并对存储在共享内存区域中的模型参数进行随机梯度更新。我们的最初实现速度极为缓慢，还依赖同步指令来确保各个处理器不会同时写入同一参数子集。实际上，每个处理器的大部分运行时间都用于等待其他处理器释放对参数写入权限的锁。

相比之下，我们采用了异步实现方式，使得每个处理器都可以随时向共享内存区域写入数据。不过，有时某个处理器对参数向量进行的更新部分内容会被另一个处理器的更新所覆盖，从而在参数更新过程中引入一定的噪声。尽管如此，这种噪声的影响似乎很小，也没有明显拖慢训练进度。

遗憾的是，大型共享内存并行计算机成本极为高昂，且其处理速度往往落后于能够通过集群连接的主流CPU。正因如此，我们能够在基于高速网络构建的集群上实现快得多的训练速度。

3.2 参数并行处理

如果该并行计算机是由多台CPU构成的网络，我们通常无法承担频繁在各个处理器之间传输所有参数的成本，因为这些参数的体积可达数十兆字节（在我们规模最大的网络中甚至接近100兆字节），通过本地网络传输会耗费大量时间。因此，我们选择对参数本身进行并行化处理，尤其是输出单元的参数，因为在我们的架构中，绝大部分计算工作都在这一环节完成。每台CPU负责计算部分输出对应的未归一化概率，并更新对应输出单元的参数（即输入到该单元的权重）。这种策略使得我们能够在几乎无需承担任何通信开销的情况下实现并行化的随机梯度上升法。实际上，这些CPU只需交换两类信息：(1) 输出softmax的归一化因子，以及(2) 隐藏层（下文用 a 表示）和词特征层（下文用 x 表示）上的梯度值。所有CPU都会重复执行在计算输出单元激活值之前的那些操作，包括词特征的选取、隐藏层激活值 a 的计算，以及相应的反向传播和参数更新步骤。不过，对于我们的网络而言，这些计算在整体计算量中所占的比例微乎其微。

例如，以在针对美联社新闻数据进行的实验中所采用的架构为例：该模型的词汇量为 $|V| = 17,964$ ；隐藏单元数量为 $h = 60$ ；模型阶数为 $n = 6$ ；词特征数量为 $m = 100$ 。处理单个训练样本时所需的全部数值运算量大约为 $|V|(1 + nm + h) + h(1 + nm) + nm$ （其中各项分别对应输出单元、隐藏单元以及词特征单元的计算量）。

feature units). In this example the fraction of the overall computation required for computing the weighted sums of the output units is therefore approximately $\frac{|V|(1+(n-1)m+h)}{|V|(1+(n-1)m+h)+h(1+(n-1)m)+(n-1)m} = 99.7\%$. This calculation is approximate because the actual CPU time associated with different operations differ, but it shows that it is generally advantageous to parallelize the output units computation. The fact that all CPUs will duplicate a very small fraction of the computations is not going to hurt the total computation time for the level of parallelization sought here, i.e. of a few dozen processors. If the number of hidden units was large, parallelizing their computation would also become profitable, but we did not investigate that approach in our experiments.

The implementation of this strategy was done on a cluster of 1.2 GHz clock-speed Athlon processors (32 x 2 CPUs) connected through a Myrinet network (a low-latency Gigabit local area network), using the MPI (Message Passing Interface) library (Dongarra et al., 1995) for the parallelization routines. The parallelization algorithm is sketched below, for a single example $(w_{t-n+1}, \dots, w_t)$, executed in parallel by CPU i in a cluster of M processors. CPU i (i ranging from 0 to $M - 1$) is responsible of a block of output units starting at number $start_i = i \times \lceil |V|/M \rceil$, the block being of length $\min(\lceil |V|/M \rceil, |V| - start_i)$.

COMPUTATION FOR PROCESSOR i , example t

1. FORWARD PHASE

- (a) Perform forward computation for the word features layer:
 $x(k) \leftarrow C(w_{t-k})$,
 $x = (x(1), x(2), \dots, x(n-1))$
- (b) Perform forward computation for the hidden layer:
 $o \leftarrow d + Hx$
 $a \leftarrow \tanh(o)$
- (c) Perform forward computation for output units in the i -th block:
 $s_i \leftarrow 0$
Loop over j in the i -th block
 - i. $y_j \leftarrow b_j + a.U_j$
 - ii. If (direct connections) $y_j \leftarrow y_j + x.W_j$
 - iii. $p_j \leftarrow e^{y_j}$
 - iv. $s_i \leftarrow s_i + p_j$
- (d) Compute and share $S = \sum_i s_i$ among the processors. This can easily be achieved with an MPI Allreduce operation, which can efficiently compute and share this sum.
- (e) Normalize the probabilities:
Loop over j in the i -th block, $p_j \leftarrow p_j / S$.
- (f) Update the log-likelihood. If w_t falls in the block of CPU $i > 0$, then CPU i sends p_{w_t} to CPU 0. CPU 0 computes $L = \log p_{w_t}$ and keeps track of the total log-likelihood.

2. BACKWARD/UPDATE PHASE, with learning rate ϵ .

- (a) Perform backward gradient computation for output units in the i -th block:
clear gradient vectors $\frac{\partial L}{\partial a}$ and $\frac{\partial L}{\partial x}$.
Loop over j in the i -th block

在此示例中，用于计算输出单元加权之和的运算量约占整体运算量的 $\frac{|V|(1+(n-1)m+h)}{\lceil |V|/M \rceil + |V| - start_i} = 99.7\%$ 。这一数值仅为估算值，因为不同运算所消耗的CPU时间存在差异，但它表明对输出单元的计算进行并行化通常具有优势。即便所有CPU仅重复极小比例的运算，对于此处所追求的数十个处理器级别的并行化程度而言，也不会对总体运算时间造成显著影响。如果隐藏单元的数量较多，对其计算进行并行化同样会带来好处，不过我们在实验中并未探讨这种方案。

该策略是在由1.2 GHz主频的Athlon处理器构成的集群上实现的，该集群包含32组各2颗CPU，这些CPU通过低延迟的千兆局域网Myrinet网络相互连接。在实现并行化流程时，开发者使用了消息传递接口标准（MPI）相关的库（Dongarra等人，1995年）。下文中将简要介绍这种并行化算法的具体思路，以单个示例 $(w_{t-n+1}, \dots, w_t)$ 为例，说明如何在由M颗处理器组成的集群中由第i颗CPU并行执行任务。其中，第i颗CPU的编号范围为0至M - 1，它负责处理从start_i = i × ⌈|V|/M⌋开始的一组输出单元，而这组单元的长度则为min(⌈|V|/M⌋, |V| - start_i)。

处理器i的计算，示例t

1. 前向阶段

- (a) 对词特征层进行前向计算： $x(k) \leftarrow C(w_{t-k})$ ， $x = (x(1), x(2), \dots, x(n-1))$ ；
- (b) 对隐藏层进行前向计算： $o \leftarrow d + Hx$ ， $a \leftarrow \tanh(o)$ ；
- (c) 对第i个块中的输出单元进行前向计算： $s_i \leftarrow 0$ 。首先遍历该块中的所有索引j： $y_j \leftarrow b_j + a$ 。
- Uii. 若存在直接连接，则 $y_j \leftarrow y_j + x$ 。
- Wiiii. $p_j \leftarrow e^{y_j}$ 。
- iv. $s_i \leftarrow s_i + p_j$ ；
- (d) 计算并在各处理器之间共享 $S = \sum_i s_i$ ，这一目标可通过MPI Allreduce操作高效实现。
- (e) 对概率值进行归一化处理：遍历第i个块中的所有索引j，计算 $p_j \leftarrow p_j / S$ 。
- (f) 更新对数似然值。若w位于CPU i对应的处理块中，则CPU i会发送出 p_w 。

通过MPI Allreduce操作可以高效地计算并共享这一求和结果。an
(e) 对各概率值进行归一化处理，即遍历第i个块中的所有索引j，计算出 $p_j \leftarrow p_j / S$ 。(f) 更新对数似然值。如果w_t落在CPU i对应的处理块内，那么CPU i就会将 p_{w_t} 发送出去。

CPU 0负责计算L = log p_{w_t} 的值，并持续记录整体的对数似然值。to

2. BACKWARD/UPDATE PHASE, with learning rate ϵ

- (a) 对第i个块中的输出单元执行反向梯度计算：先将梯度向量L a 和 $\frac{\partial L}{\partial x}$ 清零，然后遍历该块中的所有j值。

i. $\frac{\partial L}{\partial y_j} \leftarrow 1_{j==w_t} - p_j$
ii. $b_j \leftarrow b_j + \epsilon \frac{\partial L}{\partial y_j}$
If (direct connections) $\frac{\partial L}{\partial x} \leftarrow \frac{\partial L}{\partial x} + \frac{\partial L}{\partial y_j} W_j$
 $\frac{\partial L}{\partial a} \leftarrow \frac{\partial L}{\partial a} + \frac{\partial L}{\partial y_j} U_j$
If (direct connections) $W_j \leftarrow W_j + \epsilon \frac{\partial L}{\partial y_j} x$
 $U_j \leftarrow U_j + \epsilon \frac{\partial L}{\partial y_j} a$

(b) Sum and share $\frac{\partial L}{\partial x}$ and $\frac{\partial L}{\partial a}$ across processors. This can easily be achieved with an MPI Allreduce operation.

(c) Back-propagate through and update hidden layer weights:

Loop over k between 1 and h ,

$\frac{\partial L}{\partial o_k} \leftarrow (1 - a_k^2) \frac{\partial L}{\partial a_k}$
 $\frac{\partial L}{\partial x} \leftarrow \frac{\partial L}{\partial x} + H' \frac{\partial L}{\partial o}$
 $d \leftarrow d + \epsilon \frac{\partial L}{\partial o}$
 $H \leftarrow H + \epsilon \frac{\partial L}{\partial o} x'$

(d) Update word feature vectors for the input words:

Loop over k between 1 and $n - 1$

$C(w_{t-k}) \leftarrow C(w_{t-k}) + \epsilon \frac{\partial L}{\partial x(k)}$

where $\frac{\partial L}{\partial x(k)}$ is the k -th block (of length m) of the vector $\frac{\partial L}{\partial x}$.

The weight decay regularization was not shown in the above implementation but can easily be put in (by subtracting the weight decay factor times the learning rate times the value of the parameter, from each parameter, at each update). Note that parameter updates are done directly rather than through a parameter gradient vector, to increase speed, a limiting factor in computation speed being the access to memory, in our experiments.

There could be a numerical problem in the computation of the exponentials in the forward phase, whereby all the p_j could be numerically zero, or one of them could be too large for computing the exponential (step 1(c)ii above). To avoid this problem, the usual solution is to subtract the maximum of the y_j 's before taking the exponentials in the *softmax*. Thus we have added an extra Allreduce operation to share among the M processors the maximum of the y_j 's, before computing the exponentials in p_j . Let q_i be the maximum of the y_j 's in block i . Then the overall maximum $Q = \max_i q_i$ is collectively computed and shared among the M processors. The exponentials are then computed as follows: $p_j \leftarrow e^{y_j - Q}$ (instead of step 1(c)ii) to guarantee that at least one of the p_j 's will be numerically non-zero, and the maximum of the exponential's argument is 1.

By comparing clock time of the parallel version with clock time on a single processor, we found that the communication overhead was only 1/15th of the total time (for one training epoch): thus we get an almost perfect speed-up through parallelization, using this algorithm on a fast network.

On clusters with a slow network, it might be possible to still obtain an efficient parallelization by performing the communications every K examples (a *mini-batch*) rather than for each example. This requires storing K versions of the activities and gradients of the neural network in each processor. After the forward phase on the K examples, the probability sums must be shared among the

i. $\frac{\partial L}{\partial y_j} \leftarrow 1_{j==w_t} - p_j$
ii. $b_j \leftarrow b_j + \epsilon \frac{\partial L}{\partial y_j}$
If (direct connections) $\frac{\partial L}{\partial x} \leftarrow \frac{\partial L}{\partial x} + \frac{\partial L}{\partial y_j} W_j$
 $\frac{\partial L}{\partial a} \leftarrow \frac{\partial L}{\partial a} + \frac{\partial L}{\partial y_j} U_j$
If (direct connections) $W_j \leftarrow W_j + \epsilon \frac{\partial L}{\partial y_j} x$
 $U_j \leftarrow U_j + \epsilon \frac{\partial L}{\partial y_j} a$

(b) 在各个处理器之间对 $\frac{\partial L}{\partial x}$ 和 $\frac{\partial L}{\partial a}$ 进行求和并共享。这一操作可通过MPI Allreduce操作轻松实现。

(c) 通过反向传播更新隐藏层权重：

遍历从1到h的k值。

$\frac{\partial L}{\partial o_k} \leftarrow (1 - a_k^2) \frac{\partial L}{\partial a_k}$
 $\frac{\partial L}{\partial x} \leftarrow \frac{\partial L}{\partial x} + L_o \times \frac{\partial L}{\partial o}$
 $d \leftarrow d + \epsilon \frac{\partial L}{\partial o}$
 $H \leftarrow H + L_o \frac{\partial L}{\partial o} x'$

(d) 更新输入词对应的词特征向量：遍历从1到n的整数k，执

行 $C(w_{t-k}) \leftarrow C(w_{t-k}) + L_{x(k)}$ ，其中 $L_{x(k)}$ 表示向量 L_x 中长度为

m的第k个块。

上述实现中并未展示权重衰减正则化处理，但实际上很容易加入——即在每次参数更新时，从每个参数中减去权重衰减系数乘以学习率再乘以该参数当前值的数值。需要说明的是，为了提升速度，参数更新是直接进行的，而非通过参数梯度向量来完成；在我们的实验中，限制计算速度的主要因素是内存访问效率。

在前向阶段的指数运算过程中可能会出现数值问题，导致所有的 p_j 在数值上均为零，或者其中某个值过大而无法进行指数运算（即上文步骤1(c)ii所述情况）。为避免这一问题，常见的解决方法是在对数值应用softmax函数之前，先减去所有 y_j 中的最大值。因此我们在计算 p_j 中的指数之前，增加了一个额外的Allreduce操作，用于在M个处理器之间共享 y_j 的最大值。设 q_i 为第i个块中 y_j 的最大值，那么所有处理器的共同最大值 $Q = \max_i q_i$ 就会被统一计算出来并共享。此时指数的计算方式改为 $p_j \leftarrow e^{y_j - Q}$ （而非步骤1(c)ii中的方式），这样可以确保 p_j 中至少有一个数值非零，且指数运算的参数最大值为1。

通过比较并行版本与单处理器上的时钟时间，我们发现，在一次训练轮次的总耗时中，通信开销仅占其1/15；因此，在高速网络上运用该算法进行并行化处理，能够实现近乎理想的加速效果。

对于在低速网络上的集群，可以通过每隔K个样本（即一个小批量）才进行一次数据通信，而非对每个样本都通信，从而实现高效的并行化。这要求在每个处理器中存储该神经网络的K个版本的神经网络活动值与梯度值。在完成这K个样本的前向阶段后，必须在这些处理器之间共享概率总和。

processors. Then the K backward phases are initiated, to obtain the K partial gradient vectors $\frac{\partial L}{\partial \theta}$ and $\frac{\partial L}{\partial x}$. After exchanging these gradient vectors among the processors, each processor can complete the backward phase and update parameters. This method mainly saves time because of the savings in network communication latency (the amount of data transferred is the same). It may lose in convergence time if K is too large, for the same reason that batch gradient descent is generally much slower than stochastic gradient descent (LeCun et al., 1998).

4. Experimental Results

Comparative experiments were performed on the Brown corpus which is a stream of 1,181,041 words, from a large variety of English texts and books. The first 800,000 words were used for training, the following 200,000 for validation (model selection, weight decay, early stopping) and the remaining 181,041 for testing. The number of different words is 47,578 (including punctuation, distinguishing between upper and lower case, and including the syntactical marks used to separate texts and paragraphs). Rare words with frequency ≤ 3 were merged into a single symbol, reducing the vocabulary size to $|V| = 16,383$.

An experiment was also run on text from the Associated Press (AP) News from 1995 and 1996. The training set is a stream of about 14 million (13,994,528) words, the validation set is a stream of about 1 million (963,138) words, and the test set is also a stream of about 1 million (963,071) words. The original data has 148,721 different words (including punctuation), which was reduced to $|V| = 17964$ by keeping only the most frequent words (and keeping punctuation), mapping upper case to lower case, mapping numeric forms to special symbols, mapping rare words to a special symbol and mapping proper nouns to another special symbol.

For training the neural networks, the initial learning rate was set to $\epsilon_0 = 10^{-3}$ (after a few trials with a tiny data set), and gradually decreased according to the following schedule: $\epsilon_t = \frac{\epsilon_0}{1+rt}$ where t represents the number of parameter updates done and r is a decrease factor that was heuristically chosen to be $r = 10^{-8}$.

4.1 N-Gram Models

The first benchmark against which the neural network was compared is an interpolated or smoothed trigram model (Jelinek and Mercer, 1980). Let $q_t = l(freq(w_{t-1}, w_{t-2}))$ represents the discretized frequency of occurrence of the input context (w_{t-1}, w_{t-2}) .⁴ Then the conditional probability estimates have the form of a conditional mixture:

$$\hat{P}(w_t | w_{t-1}, w_{t-2}) = \alpha_0(q_t)p_0 + \alpha_1(q_t)p_1(w_t) + \alpha_2(q_t)p_2(w_t | w_{t-1}) + \alpha_3(q_t)p_3(w_t | w_{t-1}, w_{t-2})$$

with conditional weights $\alpha_i(q_t) \geq 0, \sum_i \alpha_i(q_t) = 1$. The base predictors are the following: $p_0 = 1/|V|$, $p_1(i)$ is a unigram (relative frequency of word i in the training set), $p_2(i|j)$ is the bigram (relative frequency of word i when the previous word is j), and $p_3(i|j, k)$ is the trigram (relative frequency of word i when the previous 2 words are j and k). The motivation is that when the frequency of (w_{t-1}, w_{t-2}) is large, p_3 is most reliable, whereas when it is lower, the lower-order statistics of p_2, p_1 , or even p_0 are more reliable. There is a different set of mixture weights α for each of the discrete values of q_t (which are context frequency bins). They can be easily estimated with

4. We used $l(x) = \lceil -\log((1+x)/T) \rceil$ where $freq(w_{t-1}, w_{t-2})$ is the frequency of occurrence of the input context and T is the size of the training corpus.

之后会在各处理器间交换这些梯度向量，随后每个处理器都能完成反向传播阶段并更新模型参数。该方法之所以能节省时间，主要是因为减少了网络通信延迟（尽管传输的数据总量并未改变）。但若K值过大，其收敛速度也可能会下降，原因与批量梯度下降法通常比随机梯度下降法慢得多这一情形相同（LeCun等人，1998年）。

4. 实验结果

比较实验是在Brown语料库上开展的，该语料库汇集了来自各类英文文本和书籍的1,181,041个单词。其中前800,000个单词用于训练，接下来的200,000个单词用于验证（包括模型选择、权重衰减以及提前停止策略的测试），剩余的181,041个单词则用于最终测试。该语料库中的不同单词总数为47,578个（统计范围包含标点符号、大小写区分，以及用于分隔文本和段落的句法标记）。频率为 ≤ 3 的稀有单词被合并为一个单独的符号，从而使词汇量降至 $|V| = 16,383$ 个。

此外，还针对1995年和1996年美联社的新闻文本进行了相关实验。训练集包含约1400万（13,994,528）个单词，验证集约有100万（963,138）个单词，测试集同样约为100万（963,071）个单词。原始数据中共有148,721个不同的单词（含标点符号），通过仅保留出现频率最高的单词（同时保留标点）、将大写字母转换为小写字母、把数字形式替换为特殊符号、将稀有单词统一映射为特殊符号，以及将专有名词映射为另一个特殊符号等处理方式，最终使词汇量缩减至 $|V| = 17964$ 。

在训练这些神经网络时，最初的学习率被设定为 $\epsilon_0 = 10^{-3}$ （在用极小的数据集进行几次试验后），随后会按照以下规律逐步降低： $\epsilon_t = \epsilon_0 / (1+rt)$ ，其中t表示已完成的参数更新次数，而r则是通过经验法则确定的降低系数，其值为 $r = 10^{-8}$ 。

4.1 N元语法模型

用于对比该神经网络的首个基准模型是一种插值平滑三元组模型（Jelinek和Mercer, 1980年）。设 $q_t = l(freq(w_{t-1}, w_{t-2}))$ 表示输入上下文 (w_{t-1}, w_{t-2}) 出现的离散频率。⁴ 此时，条件概率的估计值便表现为一种条件混合模型的形式。

$$\hat{P}(w_t | w_{t-1}, w_{t-2}) = \alpha_0(q_t)p_0 + \alpha_1(q_t)p_1(w_t) + \alpha_2(q_t)p_2(w_t | w_{t-1}) + \alpha_3(q_t)p_3(w_t | w_{t-1}, w_{t-2})$$

在采用条件权重的情况下，其表达式为： $\sum_i \alpha_i(q_t) \geq 0, \sum_i \alpha_i(q_t) = 1$ 。其中的基预测因子包括： $p_0 = 1/|V|$ ， $p_1(i)$ 属于单词模型（即训练集中单词i的相对出现频率）， $p_2(i|j)$ 为二元词组模型（即当前单词为i且前一个单词为j时的相对出现频率），而 $p_3(i|j, k)$ 则是三元组模型（即当前单词为i且前两个单词分别为j和k时的相对出现频率）。这样设计的理由是，当 (w_{t-1}, w_{t-2}) 的出现频率较高时， p_3 的可靠性最强；而当该频率较低时， p_2, p_1 , 甚至 p_0 的低阶统计量反而更具参考价值。对于 q_t 的每一个离散取值（也就是不同的上下文频率区间），都需要对应一组不同的混合权重，这些权重可以通过相关方法轻松估算出来。 α

4. 我们采用了 $l(x) = \lceil -\log((1+x)/T) \rceil$ 这一公式，其中 $freq(w_{t-1}, w_{t-2})$ 代表输入上下文出现的频率，T则表示训练语料库的大小。

the EM algorithm in about 5 iterations, on a set of data (the validation set) not used for estimating the unigram, bigram and trigram relative frequencies. The interpolated n-gram was used to form a mixture with the MLPs since they appear to make “errors” in very different ways.

Comparisons were also made with other state-of-the-art n-gram models: back-off n-gram models with the *Modified Kneser-Ney* algorithm (Kneser and Ney, 1995, Chen and Goodman., 1999), as well as class-based n-gram models (Brown et al., 1992, Ney and Kneser, 1993, Niesler et al., 1998). The validation set was used to choose the order of the n-gram and the number of word classes for the class-based models. We used the implementation of these algorithms in the SRI Language Modeling toolkit, described by Stolcke (2002) and in www.speech.sri.com/projects/srilm/. They were used for computing the back-off models perplexities reported below, noting that we did not give a special status to end-of-sentence tokens in the accounting of the log-likelihood, just as for our neural network perplexity. All tokens (words and punctuation) were treated the same in averaging the log-likelihood (hence in obtaining the perplexity).

4.2 Results

Below are measures of test set perplexity (geometric average of $1/\hat{P}(w_t|w_1^{t-1})$) for different models $\hat{P}$. Apparent convergence of the stochastic gradient ascent procedure was obtained after around 10 to 20 epochs for the Brown corpus. On the AP News corpus we were not able to see signs of overfitting (on the validation set), possibly because we ran only 5 epochs (over 3 weeks using 40 CPUs). Early stopping on the validation set was used, but was necessary only in our Brown experiments. A weight decay penalty of 10^{-4} was used in the Brown experiments and a weight decay of 10^{-5} was used in the APNews experiments (selected by a few trials, based on validation set perplexity). Table 1 summarizes the results obtained on the Brown corpus. All the back-off models of the table are modified Kneser-Ney n-grams, which worked significantly better than standard back-off models. When m is specified for a back-off model in the table, a class-based n-gram is used (m is the number of word classes). Random initialization of the word features was done (similarly to initialization of neural network weights), but we suspect that better results might be obtained with a knowledge-based initialization.

The **main result** is that significantly better results can be obtained when using the neural network, in comparison with the best of the n-grams, with a test perplexity difference of about 24% on Brown and about 8% on AP News, when taking the MLP versus the n-gram that worked best on the validation set. The table also suggests that the neural network was able to take advantage of more context (on Brown, going from 2 words of context to 4 words brought improvements to the neural network, not to the n-grams). It also shows that the hidden units are useful (MLP3 vs MLP1 and MLP4 vs MLP2), and that mixing the output probabilities of the neural network with the interpolated trigram always helps to reduce perplexity. The fact that simple averaging helps suggests that the neural network and the trigram make errors (i.e. low probability given to an observed word) in different places. The results do not allow to say whether the direct connections from input to output are useful or not, but suggest that on a smaller corpus at least, better generalization can be obtained without the direct input-to-output connections, at the cost of longer training: without direct connections the network took twice as much time to converge (20 epochs instead of 10), albeit to a slightly lower perplexity. A reasonable interpretation is that direct input-to-output connections provide a bit more capacity and faster learning of the “linear” part of the mapping from word features to log-

借助EM算法大约经过5次迭代，就可以在一段并未用于估算单词模型、二元词组模型以及三元组模型相对频率的数据集（即验证集）上完成处理。由于MLP模型会出现截然不同的“误差”表现，因此人们会将插值得到的n元语法模型与这些MLP模型进行混合使用。

此外，研究还将这些模型与其他最先进的n-gram模型进行了对比，包括采用改进后的Kneser-Ney算法的退避型n-gram模型（Kneser和Ney，1995年；Chen和Goodman，1999年），以及基于类别的n-gram模型（布朗等人，1992年；内伊与克内瑟，1993年；尼斯勒等人，1998年）。验证集被用来确定n-gram的阶数以及基于类别的模型所需的词类数量。我们使用了Stolcke在2002年所描述的、位于www.speech.sri.com/projects/srilm/中的SRI语言建模工具包中所实现的这些算法。这些算法被用于计算下文所述的退避型模型的困惑度，需要说明的是，在计算对数似然值时，我们并未像处理神经网络模型的困惑度那样给予句子结尾标记特殊对待。在计算对数似然值的平均值（进而得出困惑度）时，所有标记（单词和标点符号）均被一视同仁地对待。

4.2 结果

下文展示了不同模型 $\hat{P}$ 在测试集上的困惑度指标（即 $1/\hat{P}(w_t|w_1^{t-1})$ ）的几何平均值）。在Brown语料库上，通过随机梯度上升法大约经过10到20个训练轮次后便出现了明显的收敛现象。而在AP新闻语料库上，我们并未观察到过拟合的迹象（在验证集上），这可能是因为实验仅进行了5个训练轮次（使用40台CPU，耗时3周左右）。虽然采用了基于验证集的提前停止策略，但仅在Brown语料库的实验中才有必要使用该策略。Brown语料库的实验中采用了 10^{-4} 的权重衰减惩罚，而AP新闻语料库的实验则采用了 10^{-5} 的权重衰减值（该值是通过多次试验、依据验证集的困惑度来确定的）。表1汇总了在Brown语料库上获得的结果。表中所列的所有退避模型均为经过改进的Kneser-Ney n-gram模型，其性能明显优于传统的退避模型。当表中为某退避模型指定了m值时，采用的是基于类别的n-gram模型（m即为词类的数量）。词特征的初始化是随机进行的（方式与神经网络权重的初始化类似），但我们认为采用基于知识的初始化方法或许能获得更好的结果。

主要研究结果表明，与表现最佳的n-gram模型相比，使用神经网络能够获得显著更好的效果：在Brown语料库上的测试困惑度差异约为24%，而在美联社新闻语料库上的差异则为约8%，这里的比较对象是经过验证集测试后表现最好的MLP模型与对应n-gram模型。表格还显示，神经网络能够更好地利用上下文信息——在Brown语料库中，将上下文长度从2个词增加到4个词后，神经网络的性能得到提升，而n-gram模型的性能则没有变化。此外，表格表明隐藏单元的数量确实具有重要作用（如MLP3相较于MLP1，以及MLP4相较于MLP2），同时将神经网络的输出概率与插值得到的三元组模型的输出进行混合处理，始终有助于降低困惑度。简单平均处理也能起到一定作用，这说明神经网络和三元组模型会在不同位置出现错误，即对某些出现的单词赋予过低的概率。尽管现有结果无法确定输入到输出的直接连接是否有用，但至少在规模较小的语料库上，不使用这种直接连接也能实现更好的泛化能力，只不过训练时间会相应延长：没有直接连接时，网络收敛所需的时间是原来的两倍（需要20个训练周期而非10个），不过最终得到的困惑度略低。合理的解释是，输入到输出的直接连接能为从词特征到对数值的映射中的“线性”部分提供更多处理能力，并加快其学习速度。

	n	c	h	m	direct	mix	train.	valid.	test.
MLP1	5		50	60	yes	no	182	284	268
MLP2	5		50	60	yes	yes		275	257
MLP3	5		0	60	yes	no	201	327	310
MLP4	5		0	60	yes	yes		286	272
MLP5	5		50	30	yes	no	209	296	279
MLP6	5		50	30	yes	yes		273	259
MLP7	3		50	30	yes	no	210	309	293
MLP8	3		50	30	yes	yes		284	270
MLP9	5		100	30	no	no	175	280	276
MLP10	5		100	30	no	yes		265	252
Del. Int.	3						31	352	336
Kneser-Ney back-off	3							334	323
Kneser-Ney back-off	4							332	321
Kneser-Ney back-off	5							332	321
class-based back-off	3	150						348	334
class-based back-off	3	200						354	340
class-based back-off	3	500						326	312
class-based back-off	3	1000						335	319
class-based back-off	3	2000						343	326
class-based back-off	4	500						327	312
class-based back-off	5	500						327	312

Table 1: Comparative results on the Brown corpus. The deleted interpolation trigram has a test perplexity that is 33% above that of the neural network with the lowest validation perplexity. The difference is 24% in the case of the best n-gram (a class-based model with 500 word classes). *n* : order of the model. *c* : number of word classes in class-based n-grams. *h* : number of hidden units. *m* : number of word features for MLPs, number of classes for class-based n-grams. *direct*: whether there are direct connections from word features to outputs. *mix*: whether the output probabilities of the neural network are mixed with the output of the trigram (with a weight of 0.5 on each). The last three columns give perplexity on the training, validation and test sets.

probabilities. On the other hand, without those connections the hidden units form a tight bottleneck which might force better generalization.

Table 2 gives similar results on the larger corpus (AP News), albeit with a smaller difference in perplexity (8%). Only 5 epochs were performed (in approximately three weeks with 40 CPUs). The class-based model did not appear to help the n-gram models in this case, but the high-order modified Kneser-Ney back-off model gave the best results among the n-gram models.

5. Extensions and Future Work

In this section, we describe extensions to the model described above, and directions for future work.

	n	c	h	m	直接连接	mix	训练。	验证。	测试。
MLP1	5		50	60	yes	no	182	284	268
MLP2	5		50	60	yes	yes		275	257
MLP3	5		0	60	yes	no	201	327	310
MLP4	5		0	60	yes	yes		286	272
MLP5	5		50	30	yes	no	209	296	279
MLP6	5		50	30	yes	yes		273	259
MLP7	3		50	30	yes	no	210	309	293
MLP8	3		50	30	yes	yes		284	270
MLP9	5		100	30	no	no	175	280	276
MLP10模型	5		100	30	no	yes		265	252
删除插值三元组复杂度	3						31	352	336
Kneser-Ney回退机制	3							334	323
Kneser-Ney回退机制	4							332	321
Kneser-Ney回退机制	5							332	321
基于类别的回退机制	3	150						348	334
基于类别的回退机制	3	200						354	340
基于类别的回退机制	3	500						326	312
基于类别的回退机制	3	1000						335	319
基于类别的回退机制	3	2000						343	326
基于类别的回退机制	4	500						327	312
基于类别的回退机制	5	500						327	312

表1: Brown语料库上的对比测试结果。未采用插值处理的三元组模型的测试困惑度，比验证集困惑度最低的神经网络高出33%。而对于表现最佳的n-gram模型（一种拥有500个词类的基于类别的模型），这一差异为24%。各列含义如下：n：模型阶数；c：基于类别的n-gram模型中所含的词类数量；h：隐藏单元的数量；m：MLP模型的词特征数量以及基于类别的n-gram模型的词类数量；direct：表示是否存在从词特征到输出端的直接连接；mix：表示是否将神经网络的输出概率与三元组模型的输出按权重0.5进行混合。最后三列分别展示了模型在训练集、验证集和测试集上的困惑度数值。

概率值。另一方面，若没有这些连接，隐藏单元数量会形成较为狭窄的瓶颈，而这或许有助于提升模型的泛化能力。

表2基于规模更大的语料库（美联社新闻）得出了类似的结果，其困惑度差异仅为8%。实验仅进行了5轮训练（在40台CPU的条件下大约耗时三周）。在此案例中，基于类别的分类模型似乎并未对n元语法模型带来帮助，而经过高阶改进的Kneser-Ney回退机制模型则在各类n元语法模型中取得了最佳性能。

5. 扩展与未来工作

在本节中，我们将介绍对上述模型所做的扩展，以及未来研究的方向。

	n	h	m	direct	mix	train.	valid.	test.
MLP10	6	60	100	yes	yes		104	109
Del. Int.	3						126	132
Back-off KN	3						121	127
Back-off KN	4						113	119
Back-off KN	5						112	117

Table 2: Comparative results on the AP News corpus. See the previous table for the column labels.

5.1 An Energy Minimization Network

A variant of the above neural network can be interpreted as an energy minimization model following Hinton’s recent work on products of experts (Hinton, 2000). In the neural network described in the previous sections the distributed word features are used only for the “input” words and not for the “output” word (next word). Furthermore, a very large number of parameters (the majority) are expanded in the output layer: the semantic or syntactic similarities between output words are not exploited. In the variant described here, the output word is also represented by its feature vector. The network takes in input a sub-sequence of words (mapped to their feature vectors) and outputs an energy function E which is low when the words form a likely sub-sequence, high when it is unlikely. For example, the network outputs an “energy” function

$$E(w_{t-n+1}, \dots, w_t) = v \cdot \tanh(d + Hx) + \sum_{i=0}^{n-1} b_{w_{t-i}}$$

where b is the vector of biases (which correspond to unconditional probabilities), d is the vector of hidden units biases, v is the output weight vector, and H is the hidden layer weight matrix, and unlike in the previous model, input and output words contribute to x :

$$x = (C(w_t), C(w_{t-1}), C(w_{t-2}), \dots, C(w_{t-n+1})).$$

The energy function $E(w_{t-n+1}, \dots, w_t)$ can be interpreted as an unnormalized log-probability for the joint occurrence of $(w_{t-n+1}, \dots, w_t)$. To obtain a conditional probability $\hat{P}(w_t | w_{t-n+1}^{t-1})$ it is enough (but costly) to normalize over the possible values of w_t , as follows:

$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1}) = \frac{e^{-E(w_{t-n+1}, \dots, w_t)}}{\sum_i e^{-E(w_{t-n+1}, \dots, w_{t-1}, i)}}$$

Note that the total amount of computation is comparable to the architecture presented earlier, and the number of parameters can also be matched if the v parameter is indexed by the identity of the target word (w_t). Note that only b_{w_t} remains after the above softmax normalization (any linear function of the w_{t-i} for $i > 0$ is canceled by the softmax normalization). As before, the parameters of the model can be tuned by stochastic gradient ascent on $\log \hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})$, using similar computations.

In the products-of-experts framework, the hidden units can be seen as the experts: the joint probability of a sub-sequence $(w_{t-n+1}, \dots, w_t)$ is proportional to the exponential of a sum of terms associated with each hidden unit j , $v_j \tanh(d_j + H_j x)$. Note that because we have chosen to decompose the probability of a whole sequence in terms of conditional probabilities for each element,

	n	h	m	直接连接	mix	训练。	验证。	测试。
MLP10模型	6	60	100	yes	yes		104	109
删除插值三元组冗余度	3						126	132
回退KN策略	3						121	127
回退KN策略	4						113	119
回退KN策略	5						112	117

表2: 映射C 基于AP新闻语料库的对比结果。列标题详见上表。s.

5.1 能量最小化网络

上述神经网络的一种变体可被视作一种能量最小化模型，这一观点源自Hinton近期关于专家乘积模型的研究成果（Hinton，2000年）。在前面章节所描述的神经网络中，分布式词特征仅用于“输入”词，而未被用于“输出”词（即下一个词）。此外，绝大多数参数都集中在输出层，因此并未利用输出词之间的语义或句法相似性。而在本文所述的变体中，输出词同样通过其特征向量来表示。该网络会接收由词组成的子序列作为输入（这些词会被映射为对应的特征向量），并输出一个能量函数E——当这些词构成一个合理的子序列时，该函数的值较低；反之则较高。例如，该网络就会输出这样一个‘能量’函数。

$$E(w_{t-n+1}, \dots, w_t) = v \cdot \tanh(d + Hx) + \sum_{i=0}^{n-1} b_{w_{t-i}}$$

其中， b 为偏置参数向量（对应无条件概率）， d 为隐藏单元数量的偏置参数向量， v 为输出权重向量， H 为隐藏层权重矩阵；与之前的模型不同，输入词和输出词也会对能量值产生影响。

$$x = (C(w_t), C(w_{t-1}), C(w_{t-2}), \dots, C(w_{t-n+1})).$$

能量函数 $E(w_{t-n+1}, \dots, w_t)$ 可被理解为 $(w_{t-n+1}, \dots, w_t)$ 共同出现的未归一化的对数概率。若要得到条件概率 $\hat{P}(w_t | w_{t-n+1}^{t-1})$ ，只需对 w_t 的所有可能取值进行归一化处理，尽管这一操作的计算成本较高。

$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1}) = \frac{e^{-E(w_{t-n+1}, \dots, w_t)}}{\sum_i e^{-E(w_{t-n+1}, \dots, w_{t-1}, i)}}$$

需要注意的是，其总计算量与之前介绍的架构相当；如果让 v 参数根据目标词的身份（即 w_t ）来索引，参数的数量也可以保持一致。此外，在经过上述softmax归一化后，仅剩下 b_{w_t} ，因为对于任意 $i > 0$ 而言， w_{t-i} 的线性函数都会被softmax归一化抵消掉。与以往相同，该模型的参数可以通过对 $\log \hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})$ 应用随机梯度上升法并进行类似计算来进行调整。

在专家乘积框架中，这些隐藏单元可被视为相应的专家：子序列 $(w_{t-n+1}, \dots, w_t)$ 的联合概率与每个隐藏单元 j 对应的各项之和的指数值成正比，即 $v_j \tanh(d_j + H_j x)$ 。需要注意的是，由于我们选择将整个序列的概率分解为各个元素的条件概率，因此梯度计算变得十分简便。

the computation of the gradient is tractable. This is not the case for example with products-of-HMMs (Brown and Hinton, 2000), in which the product is over experts that view the whole sequence, and which can be trained with approximate gradient algorithms such as the contrastive divergence algorithm (Brown and Hinton, 2000). Note also that this architecture and the products-of-experts formulation can be seen as extensions of the very successful **Maximum Entropy** models (Berger et al., 1996), but where the basis functions (or “features”, here the hidden units activations) are learned by penalized maximum likelihood at the same time as the parameters of the features linear combination, instead of being learned in an outer loop, with greedy feature subset selection methods.

We have implemented and experimented with the above architecture, and have developed a speed-up technique for the neural network training, based on importance sampling and yielding a 100-fold speed-up (Bengio and Senécal, 2003).

Out-of-vocabulary words. An advantage of this architecture over the previous one is that it can easily deal with out-of-vocabulary words (and even assign them a probability!). The main idea is to first guess an initial feature vector for such a word, by taking a weighted convex combination of the feature vectors of other words that could have occurred in the same context, with weights proportional to their conditional probability. Suppose that the network assigned a probability $\hat{P}(i|w_{t-n+1}^{t-1})$ to words $i \in V$ in context w_{t-n+1}^{t-1} , and that in this context we observe a new word $j \notin V$. We initialize the feature vector $C(j)$ for j as follows: $C(j) \leftarrow \sum_{i \in V} C(i) \hat{P}(i|w_{t-n+1}^{t-1})$. We can then incorporate j in V and re-compute probabilities for this slightly larger set (which only requires a renormalization for all the words, except for word i , which requires a pass through the neural network). This feature vector $C(i)$ can then be used in the input context part when we try to predict the probabilities of words that follow word i .

5.2 Other Future Work

There are still many challenges ahead to follow-up on this work. In the short term, methods to speed-up training and recognition need to be designed and evaluated. In the longer term, more ways to generalize should be introduced, in addition to the two main ways exploited here. Here are some ideas that we intend to explore:

- 1. Decomposing the network in sub-networks, for example using a clustering of the words. Training many smaller networks should be easier and faster.
- 2. Representing the conditional probability with a tree structure where a neural network is applied at each node, and each node represents the probability of a word class given the context and the leaves represent the probability of words given the context. This type of representation has the potential to reduce computation time by a factor $|V|/\log |V|$ (see Bengio, 2002).
- 3. Propagating gradients only from a subset of the output words. It could be the words that are conditionally most likely (based on a faster model such as a trigram, see Schwenk and Gauvain, 2002, for an application of this idea), or it could be a subset of the words for which the trigram has been found to perform poorly. If the language model is coupled to a speech recognizer, then only the scores (unnormalized probabilities) of the acoustically ambiguous words need to be computed. See also Bengio and Senécal (2003) for a new accelerated training method using importance sampling to select the words.

与之不同的是，例如基于HMM的专家乘积模型（Brown和Hinton，2000）中，其概率计算需要遍历所有能够观察整个序列的专家，且这类模型通常需借助对比散度算法（Brown和Hinton，2000）等近似梯度算法才能进行训练。此外，这种架构以及专家乘积建模方法实际上可以视为极为成功的最大熵模型（Berger等人，1996）的扩展形式——只不过在这里，基函数（或称“特征”，即隐藏单元的激活值）是与特征线性组合的参数一同通过带惩罚项的最大似然方法来学习的，而非像传统方法那样通过贪婪特征子集选择方法在独立的外循环中逐步学习。

我们实现了上述架构并进行了相关实验，同时基于重要性采样开发出一种用于加速神经网络训练的加速技术，该技术可使训练速度提升100倍（本吉奥与塞内卡尔，2003年）。

词汇表外单词。相较于之前的架构，这种架构的优势在于能够轻松处理词汇表外单词（甚至还能为它们赋予概率值！）。其核心思路是首先为这类单词估算一个初始特征向量，方法是将可能在相同上下文中出现的其他单词的特征向量进行加权凸组合，权重则按照这些单词的条件概率来确定。假设网络在上下文 w_{t-n+1}^{t-1} 中为单词 $i \in V$ 分配了概率 $\hat{P}(i|w_{t-n+1}^{t-1})$ ，而在该上下文中又出现了一个新的单词 $j \notin V$ 。此时我们可以按如下方式初始化 j 的特征向量 $C(j)$ ： $C(j) \leftarrow \sum_{i \in V} C(i) \hat{P}(i|w_{t-n+1}^{t-1})$ 。之后就可以将 j 纳入 V 中，并为这个规模稍大的集合重新计算概率值（这仅需对所有单词进行归一化处理，单词 i 则需要再次通过神经网络计算一次）。之后在尝试预测单词 i 之后的单词的概率时，就可以使用这个特征向量 $C(i)$ 。

5.2 其他未来工作

要在此项工作基础上进一步深入研究，仍面临诸多挑战。短期内需要设计并评估能够加快训练与识别速度的方法；而从更长远的视角来看，除了目前已采用的两种主要方法之外，还需探索更多提升模型泛化能力的途径。以下是我们计划探讨的一些思路：

- 1. 将神经网络拆分为多个子网络，例如通过对词汇进行聚类来实现。这样处理后，训练众多较小的网络会更为简便且高效。
- 2. 采用树形结构来表示条件概率，即在每个节点上应用神经网络——其中每个节点代表在给定上下文条件下某个词类的概率，而叶节点则表示在相同上下文条件下各单词的出现概率。这种表示方式有望将计算时间减少 $|V|/\log |V|$ 倍（参见本吉奥，2002年）。
- 3. 仅从部分输出词中传播梯度。这些词可以是条件概率最高的词（基于更快速的模型，如三元组模型，有关该思路的应用可参见Schwenk与Gauvain在2002年的研究），也可以是那些经检测使用三元组模型效果不佳的词子集。如果该语言模型与语音识别器相连，则仅需计算那些在声学上存在歧义的词的分（未标准化的概率值）。另外，Bengio与Senécal在2003年提出了一种新的加速训练方法，该方法利用重要性采样来筛选所需处理的词。

- 4. Introducing a-priori knowledge. Several forms of such knowledge could be introduced, such as: semantic information (e.g., from WordNet, see Fellbaum, 1998), low-level grammatical information (e.g., using parts-of-speech), and high-level grammatical information, e.g., coupling the model to a stochastic grammar, as suggested in Bengio (2002). The effect of longer term context could be captured by introducing more structure and parameter sharing in the neural network, e.g. using time-delay or recurrent neural networks. In such a multi-layered network the computation that has been performed for small groups of consecutive words does not need to be redone when the network input window is shifted. Similarly, one could use a recurrent network to capture potentially even longer term information about the subject of the text.
- 5. Interpreting (and possibly using) the word feature representation learned by the neural network. A simple first step would start with $m = 2$ features, which can be more easily displayed. We believe that more meaningful representations will require large training corpora, especially for larger values of m .
- 6. Polysemous words are probably not well served by the model presented here, which assigns to each word a single point in a continuous semantic space. We are investigating extensions of this model in which each word is associated with multiple points in that space, each associated with the different senses of the word.

6. Conclusion

The experiments on two corpora, one with more than a million examples, and a larger one with above 15 million words, have shown that the proposed approach yields much better perplexity than a state-of-the-art method, the smoothed trigram, with differences between 10 and 20% in perplexity.

We believe that the main reason for these improvements is that the proposed approach allows to take advantage of the learned distributed representation to fight the curse of dimensionality with its own weapons: each training sentence informs the model about a combinatorial number of other sentences.

There is probably much more to be done to improve the model, at the level of architecture, computational efficiency, and taking advantage of prior knowledge. An important priority of future research should be to improve speed-up techniques⁵ as well as ways to increase capacity without increasing training time too much (to deal with corpora with hundreds of millions of words or more). A simple idea to take advantage of temporal structure and extend the size of the input window to include possibly a whole paragraph (without increasing too much the number of parameters or computation time) is to use a time-delay and possibly recurrent neural networks. Evaluations of the type of models presented here in applicative contexts would also be useful, but see work already done by Schwenk and Gauvain (2002) for improvements in speech recognition word error rate.

More generally, the work presented here opens the door to improvements in statistical language models brought by replacing “tables of conditional probabilities” by more compact and smoother representations based on distributed representations that can accommodate far more conditioning variables. Whereas much effort has been spent in statistical language models (e.g. stochastic grammars) to restrict or summarize the conditioning variables in order to avoid overfitting, the type of

5. See work by Bengio and Senécal (2003) for a 100-fold speed-up technique.

4. 引入先验知识。此类知识有多种表现形式，例如：语义信息（可从WordNet获取，相关研究可见Fellbaum在1998年的成果）、低层级语法信息（如通过词性标注实现），以及高层级语法信息，比如按照Bengio在2002年提出的建议将模型与随机语法相结合。为了捕捉更长时序的上下文信息，可以在神经网络中加入更多结构并实现参数共享，例如采用时间延迟结构或循环神经网络。在这样的多层网络中，当网络输入窗口发生偏移时，无需重新计算针对连续少量词组已完成的运算。同样地，也可以利用循环神经网络来捕获与文本主题相关的更长时序的信息。

5. 解读（并可能应用）神经网络所学到的词特征表示。一个简单的初步做法是从 $= 2$ 个特征开始，这样的特征更易于呈现。我们认为，若要获得更具意义的表示，就需要规模更大的训练语料库，尤其是当特征数量m较大时。

6. 对于多义词而言，当前这种为每个词在连续语义空间中仅分配一个点的模型可能无法很好地处理这类词汇。我们正在研究该模型的扩展版本，在这种版本中，每个词都会在该空间中被关联多个点，而这些点分别对应该词的不同含义。

6. 结论

在包含超过一百万条样本的语料库以及包含超过1500万个单词的更大规模语料库上进行的实验表明，所提出的方法相比现有的最先进方法——即平滑三词组模型——能显著降低困惑度，两者之间的困惑度差异在10%到20%之间。

我们认为，这些改进的主要原因在于所提出的方法能够充分利用所学到的分布式表示来应对维度灾难，而其自身也具备相应的解决手段：每条训练句子都会为模型提供关于其他大量句子的信息。

要想进一步提升该模型，还有许多工作有待开展，具体涉及架构设计、计算效率的提升以及先验知识的有效利用等方面。未来研究的一个重要方向应是改进加速技术⁵，同时探索在不大幅增加训练时间的情况下提升模型容量（以便处理包含数亿词甚至更多词的文本语料库）。一个简单的思路是利用时间延迟机制以及循环神经网络，从而充分利用时间结构并适当扩大输入窗口的范围，甚至纳入整段文本，同时尽量不增加过多的参数数量或计算成本。此外，在实际应用场景中对这类模型进行评估也会很有价值；至于在降低语音识别单词错误率方面的相关成果，可参考Schwenk与Gauvain在2002年发表的研究。

更一般而言，本文所介绍的研究为统计语言模型的改进打开了新途径——通过采用基于分布式表示的更为紧凑且平滑的映射C，来替代传统的“条件概率表”，从而能够容纳更多的条件变量。在统计语言模型领域（例如随机文法），人们已经投入了大量精力去限制或简化条件变量以避免过拟合，而此处描述的模型类型则将挑战转移到了其他方面：虽然需要执行更多的计算操作，但其计算量和内存需求仅与条件变量的数量呈线性增长，而非指数级上升。

5. 如需了解基于Bengio与Senécal在2003年提出的加速技术，可实现100倍的性能提升，可参考相关研究。

models described here shifts the difficulty elsewhere: many more computations are required, but computation and memory requirements scale linearly, not exponentially with the number of conditioning variables.

ACKNOWLEDGMENTS

The authors would like to thank Léon Bottou, Yann Le Cun and Geoffrey Hinton for useful discussions. This research was made possible by funding from the NSERC granting agency, as well as the MITACS and IRIS networks.

References

D. Baker and A. McCallum. Distributional clustering of words for text classification. In *SIGIR’98*, 1998.

J.R. Bellegarda. A latent semantic analysis framework for large-span language modeling. In *Proceedings of Eurospeech 97*, pages 1451–1454, Rhodes, Greece, 1997.

S. Bengio and Y. Bengio. Taking on the curse of dimensionality in joint distributions using neural networks. *IEEE Transactions on Neural Networks, special issue on Data Mining and Knowledge Discovery*, 11(3):550–557, 2000a.

Y. Bengio. New distributed probabilistic language models. Technical Report 1215, Dept. IRO, Université de Montréal, 2002.

Y. Bengio and S. Bengio. Modeling high-dimensional discrete data with multi-layer neural networks. In S. A. Solla, T. K. Leen, and K-R. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12, pages 400–406. MIT Press, 2000b.

Y. Bengio and J-S. Senécal. Quick training of probabilistic neural nets by importance sampling. In *AISTATS*, 2003.

A. Berger, S. Della Pietra, and V. Della Pietra. A maximum entropy approach to natural language processing. *Computational Linguistics*, 22:39–71, 1996.

A. Brown and G.E. Hinton. Products of hidden markov models. Technical Report GCNU TR 2000-004, Gatsby Unit, University College London, 2000.

P.F. Brown, V.J. Della Pietra, P.V. DeSouza, J.C. Lai, and R.L. Mercer. Class-based *n*-gram models of natural language. *Computational Linguistics*, 18:467–479, 1992.

S.F. Chen and J.T. Goodman. An empirical study of smoothing techniques for language modeling. *Computer, Speech and Language*, 13(4):359–393, 1999.

S. Deerwester, S.T. Dumais, G.W. Furnas, T.K. Landauer, and R. Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407, 1990.

J. Dongarra, D. Walker, and The Message Passing Interface Forum. MPI: A message passing interface standard. Technical Report <http://www-unix.mcs.anl.gov/mpi>, University of Tennessee, 1995.

这类模型将难题转移到了其他方面：尽管需要更多的计算资源，但计算量和内存需求会随着条件变量数量的增加而呈线性增长，不会出现指数级上升的情况。

致谢

作者们想要感谢Léon Bottou、Yann Le Cun以及Hinton，感谢他们带来的有益讨论。本研究得以开展，得益于加拿大自然科学与工程研究委员会、加拿大数学、计算机科学及应用技术协会以及IRIS网络的资助。

参考文献

D. Baker与A. McCallum。用于文本分类的单词分布聚类方法。载于1998年召开的SIGIR’ 98会议论文集。J.R. Bellegarda。一种用于大跨度语言建模的潜在语义分析框架。载于1997年在希腊罗德斯岛召开的Eurospeech 97会议论文集，第1451–1454页。S. Bengio与Y. Bengio。利用神经网络应对联合分布中的维度灾难问题。《IEEE神经网络汇刊》，数据挖掘与知识发现专题特刊，11(3):550–557，2000a年。Y. Bengio。新的分布式概率语言模型。蒙特利尔大学信息处理研究所技术报告1215号，2002年。Y. Bengio与S. Bengio。运用多层神经网络对高维离散数据进行建模。载于S. A. Solla、T. K. Leen和K-R. Müller主编的《神经信息处理系统进展》第12卷，第400–406页，MIT出版社，2000b年。Y. Bengio与J-S. Senécal。通过重要性采样实现概率神经网络的快速训练。载于2003年召开的AISTATS会议论文集。A. Berger、S. Della Pietra与V. Della Pietra。自然语言处理的最大熵方法。《计算语言学》，22:39–71，1996年。A. Brown与G.E. Hinton。隐马尔可夫模型的乘积形式。伦敦大学学院加茨比单元技术报告GCNU TR 2000-004号，2000年。P.F. Brown、V.J. Della Pietra、P.V. DeSouza、J.C. Lai与R.L. Mercer。基于类别的n-gram自然语言模型。《计算语言学》，18:467–479，1992年。S.F. Chen与J.T. Goodman。语言建模中平滑技术的实证研究。《计算机、语音与语言》，13(4):359–393，1999年。S. Deerwester、S.T. Dumais、G.W. Furnas、T.K. Landauer与R. Harshman。基于潜在语义分析的索引技术。《美国信息科学学会期刊》，41(6):391–407，1990年。J. Dongarra、D. Walker与消息传递接口论坛。MPI：一种消息传递接口标准。田纳西大学技术报告<http://www-unix.mcs.anl.gov/mpi>，1995年。

J.L. Elman. Finding structure in time. *Cognitive Science*, 14:179–211, 1990.

C. Fellbaum. *WordNet: An Electronic Lexical Database*. MIT Press, 1998.

J. Goodman. A bit of progress in language modeling. Technical Report MSR-TR-2001-72, Microsoft Research, 2001.

G.E. Hinton. Learning distributed representations of concepts. In *Proceedings of the Eighth Annual Conference of the Cognitive Science Society*, pages 1–12, Amherst 1986, 1986. Lawrence Erlbaum, Hillsdale.

G.E. Hinton. Training products of experts by minimizing contrastive divergence. Technical Report GCNU TR 2000-004, Gatsby Unit, University College London, 2000.

F. Jelinek and R. L. Mercer. Interpolated estimation of Markov source parameters from sparse data. In E. S. Gelsema and L. N. Kanal, editors, *Pattern Recognition in Practice*. North-Holland, Amsterdam, 1980.

K.J. Jensen and S. Riis. Self-organizing letter code-book for text-to-phoneme neural network model. In *Proceedings ICSLP*, 2000.

S.M. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, ASSP-35 (3):400–401, March 1987.

R. Kneser and H. Ney. Improved backing-off for m-gram language modeling. In *International Conference on Acoustics, Speech and Signal Processing*, pages 181–184, 1995.

Y. LeCun, L. Bottou, G.B. Orr, and K.-R. Müller. Efficient backprop. In G.B. Orr and K.-R. Müller, editors, *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998.

R. Miikkulainen and M.G. Dyer. Natural language processing with modular neural networks and distributed lexicon. *Cognitive Science*, 15:343–399, 1991.

H. Ney and R. Kneser. Improved clustering techniques for class-based statistical language modelling. In *European Conference on Speech Communication and Technology (Eurospeech)*, pages 973–976, Berlin, 1993.

T.R. Niesler, E.W.D. Whittaker, and P.C. Woodland. Comparison of part-of-speech and automatically derived category-based language models for speech recognition. In *International Conference on Acoustics, Speech and Signal Processing*, pages 177–180, 1998.

A. Paccanaro and G.E. Hinton. Extracting distributed representations of concepts and relations from positive and negative propositions. In *Proceedings of the International Joint Conference on Neural Network, IJCNN’2000*, Como, Italy, 2000. IEEE, New York.

F. Pereira, N. Tishby, and L. Lee. Distributional clustering of english words. In *30th Annual Meeting of the Association for Computational Linguistics*, pages 183–190, Columbus, Ohio, 1993.

J.L. Elman. 《在时间中寻找结构》。《认知科学》，14卷：179–211页，1990年。C. Fellbaum. 《WordNet：一种电子词典数据库》。MIT出版社，1998年。J. Goodman. 《语言建模领域的一些进展》。微软研究院技术报告MSR-TR-2001-72，2001年。G.E. Hinton. 《学习概念的分布式表示》。载于《认知科学学会第八届年会论文集》，第1–12页，阿默斯特，1986年。Lawrence Erlbaum出版社，希尔斯代尔。G.E. Hinton. 《通过最小化对比散度来训练专家乘积模型》。伦敦大学学院加茨比单元技术报告GCNU TR 2000-004，2000年。F. Jelinek与R. L. Mercer. 《基于稀疏数据的马尔可夫源参数插值估计》。载于E. S. Gelsema与L. N. Kanal主编的《实践中的模式识别》一书，北荷兰出版社，阿姆斯特丹，1980年。K.J. Jensen与S. Riis. 《用于文本到音素神经网络模型的自组织字母码本》。载于ICSLP会议论文集，2000年。S.M. Katz. 《为语音识别器的语言模型组件从稀疏数据中估计概率》。《IEEE声学、语音与信号处理汇刊》，ASSP-35 (3)：400–401页，1987年3月。R. Kneser与H. Ney. 《改进的m-gram语言建模退火算法》。载于国际声学、语音与信号处理会议论文集，第181–184页，1995年。Y. LeCun、L. Bottou、G.B. Orr与K.-R. Müller. 《高效反向传播算法》。载于G.B. Orr与K.-R. Müller主编的《神经网络：行业技巧》一书，第9–50页。施普林格出版社，1998年。R. Miikkulainen与M.G. Dyer. 《利用模块化神经网络与分布式词汇表进行自然语言处理》。《认知科学》，15卷：343–399页，1991年。H. Ney与R. Kneser. 《面向基于类别的统计语言建模的改进聚类技术》。载于欧洲语音通信与技术会议（Eurospeech）论文集，第973–976页，柏林，1993年。T.R. Niesler、E.W.D. Whittaker与P.C. Woodland. 《语音识别中词性标注模型与自动生成的基于类别的语言模型之比较》。载于国际声学、语音与信号处理会议论文集，第177–180页，1998年。A. Paccanaro与G.E. Hinton. 《从正面与负面命题中提取概念及关系的分布式表示》。载于2000年在意大利科莫举行的国际联合神经网络会议IJCNN’2000论文集，IEEE出版社，纽约。F. Pereira、N. Tishby与L. Lee. 《英语单词的分布式聚类》。载于第30届计算语言学协会年会论文集，第183–190页，俄亥俄州哥伦布市，1993年。

S. Riis and A. Krogh. Improving protein secondary structure prediction using structured neural networks and multiple sequence profiles. *Journal of Computational Biology*, pages 163–183, 1996.

J. Schmidhuber. Sequential neural text compression. *IEEE Transactions on Neural Networks*, 7(1): 142–146, 1996.

H. Schutze. Word space. In S. J. Hanson, J. D. Cowan, and C. L. Giles, editors, *Advances in Neural Information Processing Systems 5*, pages pp. 895–902, San Mateo CA, 1993. Morgan Kaufmann.

H. Schwenk and J-L. Gauvain. Connectionist language modeling for large vocabulary continuous speech recognition. In *International Conference on Acoustics, Speech and Signal Processing*, pages 765–768, Orlando, Florida, 2002.

A. Stolcke. SRILM - an extensible language modeling toolkit. In *Proceedings of the International Conference on Statistical Language Processing*, Denver, Colorado, 2002.

W. Xu and A. Rudnicky. Can artificial neural network learn language models. In *International Conference on Statistical Language Processing*, pages M1–13, Beijing, China, 2000.

S. Riis与A. Krogh在1996年的《Journal of Computational Biology》上发表文章，探讨了如何利用结构化神经网络与多序列谱来提升蛋白质二级结构预测的准确性。J. Schmidhuber于1996年在《IEEE神经网络汇刊》的第7卷第1期中提出了序列神经网络文本压缩方法，相关论文页码为142–146。H. Schutze在1993年由S. J. Hanson、J. D. Cowan和C. L. Giles主编的《Advances in Neural Information Processing Systems 5》中论述了词空间概念，该文页码为895–902，出版地为加利福尼亚州圣马特奥。Morgan Kaufmann、H. Schwenk与J-L. Gauvain在2002年于佛罗里达州奥兰多举行的国际声学、语音与信号处理会议上发表了关于基于连接主义的大词汇量连续语音识别语言建模的研究，相关论文页码为765–768。A. Stolcke在2002年于科罗拉多州丹佛举行的国际统计语言处理会议上传播了SRILM这一可扩展的语言建模工具包。最后，W. Xu与A. Rudnicky在2000年于中国北京举行的国际统计语言处理会议上探讨了人工神经网络是否具备学习语言模型的能力，相关论文编号为M1–13。